Available online at www.sciencedirect.com
ScienceDirect
Comput. Methods Appl. Mech. Engrg. 416 (2023) 116298
Computer methods in applied mechanics and engineering
www.elsevier.com/locate/cma

Hyper-reduction for Petrov–Galerkin reduced order models

S. Ares de Parga[a,b,*], J.R. Bravo[a,b], J.A. Hernández[a,b,c], R. Zorrilla[a,b], R. Rossi[a,b]

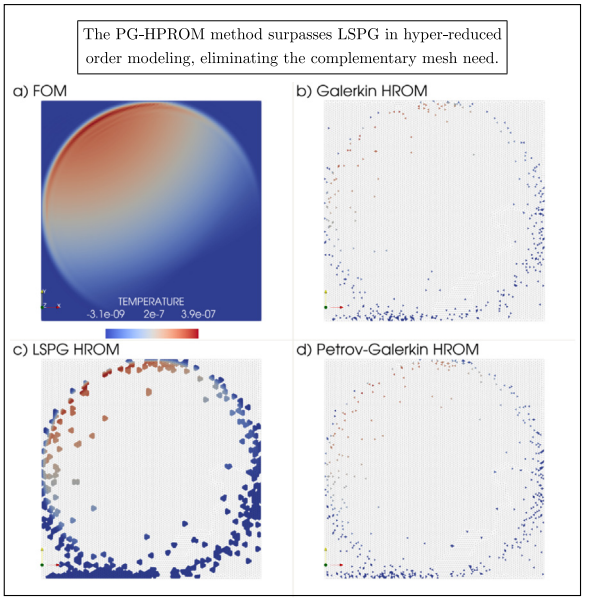
[a] *Universitat Politècnica de Catalunya, Department of Civil and Environmental Engineering, Building B0, Campus Nord, Jordi Girona 1-3, Barcelona 08034, Spain*

[b] *Centre Internacional de Mètodes Numèrics en Enginyeria (CIMNE), Universitat Politècnica de Catalunya, Building C1, Campus Nord, Jordi Girona 1-3, Barcelona 08034, Spain*

[c] *Universitat Politècnica de Catalunya, E.S. d'Enginyeries Industrial, Aeroespacial i Audiovisual de Terrassa, C/ Colom, 11, Terrassa 08222, Spain*

Received 27 March 2023; received in revised form 22 June 2023; accepted 20 July 2023
Available online 26 August 2023

Graphical Abstract



Abstract

Projection-based Reduced Order Models are based on the idea of minimizing the discrete residual of a "full order model" (FOM) while at the same time constraining the unknowns to live in a space of reduced dimension. For problems with symmetric

* Corresponding author at: Universitat Politècnica de Catalunya, Department of Civil and Environmental Engineering, Building B0, Campus Nord, Jordi Girona 1-3, Barcelona 08034, Spain.
E-mail address: sebastian.ares@upc.edu (S. Ares de Parga).

https://doi.org/10.1016/j.cma.2023.116298

positive definite (SPD) Jacobians, this minimization can be achieved optimally by projecting the full order residual onto the approximation basis (Galerkin Projection). This approach is sub-optimal for problems with non-SPD Jacobians since it only guarantees that the projection of the residual onto the chosen basis is minimized and not the residual itself. One possible alternative in such cases is to directly minimize the 2-norm of the residual. This minimization can be achieved either by using QR factorization to solve the resulting least-squares problem or by employing the method of the normal equations (LSPG) to the same end. The first approach involves constructing and factorizing a rectangular tall and skinny matrix of size proportional to the number of unknowns of the FOM. The LSPG method avoids the use of the large matrix by directly constructing the product by its transpose. Unfortunately, constructing such a product element by element is not feasible and requires the use of a complementary mesh, which adds an extra layer of complexity to the hyper-reduction process when performing mesh sampling. The main idea of this work is to propose an alternative technique based on the idea of Petrov–Galerkin minimization. Essentially, we choose a left basis so that a least-squares minimization procedure can be carried out on a reduced problem while guaranteeing that the discrete full order residual is minimized. The resulting procedure is applicable to problems with both SPD and non-SPD Jacobians. Additionally, the resulting minimization problem can be assembled element by element, avoiding the use of the complementary mesh and simplifying implementation in the context of finite elements. The resulting technique is amenable to hyper-reduction by the use of the Empirical Cubature Method and can be readily applied in the context of nonlinear reduction procedures.

Keywords: Petrov–Galerkin; LSPG; Reduced Order Models; Hyper-reduction

1. Introduction

Projection-based reduced order models (PROM) can significantly reduce the time and storage required for the solution of high-fidelity (high-dimensional) discretized PDE based computational models for a given range of parametric settings, also known as full-order models (FOM) in the literature. These PROMs are intended for the creation of time-critical models such as digital twins, design/shape optimization, control problem optimization, and other applications. Unfortunately, the cost of creating a PROM of dimension n scales with both n and the dimension of the underlying FOM $N \gg n$. To overcome this problem, one popular approach is to divide the computation into two parts: one that scales with the FOM but can be executed offline to compute some numerical quantities; and another that uses the aforementioned numerical quantities to perform all online computations with a complexity that is independent of the FOM's large dimension. The first part of this offline–online decomposition is at the heart of model order reduction, where the state of a large-scale system resides on a lower dimensional manifold engendered by time evolution and/or input parameter variation. To do this, solution snapshots corresponding to a carefully chosen set of input parameters are subjected to dimensionality reduction (compression) in order to determine a reduced-order basis (ROB), using, for instance, Proper Orthogonal Decomposition (POD) [1,2]. The FOM is then projected onto the subspace spanned by this ROB in order to provide rapid and accurate online numerical predictions. This offline–online technique has been shown to be effective for parametric, linear problems [3,4], as well as nonlinear problems with a low-order polynomial dependency [5,6].

Similarly, computational methods known as hyper-reduction have gained popularity in overcoming computational bottlenecks caused by repeated re-evaluations of parametric reduced-order operators. These methods approximate these operators with a computational cost that is independent of the FOM, trading some of the PROM's accuracy for speed. Carlberg first introduced the classification of hyper-reduction methods into two types: the approximate-then-project and the project-then-approximate, as described in [7]. The approximate-then-project hyper-reduction methods approximate first an operator of interest and then project the approximation onto the left ROB. The underlying idea for avoiding a computational cost that grows with the problem's large dimension N may be traced back to the gappy POD technique, which was initially devised for image reconstruction [8]. After nearly a decade, the empirical interpolation method was launched on a continuous level with its discrete variant known as discrete EIM (DEIM) [9], which is likely the most popular approximate-then-project hyper-reduction method to date. Other approximate-then-project hyper-reduction methods have been introduced, such as the missing point estimation [10] and the collocation method [11]. Another noteworthy approximate-then-project hyper-reduction method is the Gauss–Newton with approximated tensors (GNAT) method [12], which was conceived for the Petrov–Galerkin projection rather than the Galerkin framework. On the other hand, hyper-reduction methods of the project-then-approximate type approximate directly the projection onto the left ROB. They can be regarded as extended quadrature rules, in which the set of

quadrature "points" and associated weights are learned via a supervised method on an empirical set of training data. Among this family of methods, we can find the energy-conserving sampling and weighting (ECSW) method developed in Ref. [13], and the empirical cubature method (ECM) [14,15]. These methods compute a subset of the underlying FOM's elements (or other entities) that define the quadrature points, then construct an approximation of the full solution, which is commonly known as mesh sampling.

Hyper-reduction methods have been widely demonstrated to produce numerically stable hyper-reduced PROMs (HPROM) [9–11,13–17], especially for standard Galerkin PROM. By definition, in a Galerkin projection, the left ($\boldsymbol{\Psi}$) and right ($\boldsymbol{\Phi}$) ROBs are set to be equal, or the test and approximation spaces are the same ($\boldsymbol{\Psi} = \boldsymbol{\Phi}$). The first methods that explored the acceleration of Petrov–Galerkin (PG) PROM, where the left and right ROBs differ ($\boldsymbol{\Psi} \neq \boldsymbol{\Phi}$), are of the approximate-then-project type. These include a gappy-POD-like method, similar to DEIM, called the Gauss–Newton with approximated tensors (GNAT) method [7,12,18,19], and a least-squares variant of the collocation method [11]. The GNAT was developed in the context of PG-PROMs, especially for PG-PROMs in which the left ROB is chosen to minimize the discrete, nonlinear residual over the approximation subspace associated with the right ROB in the 2-norm. In this scenario, applying Newton's technique to solve the system of equations is equal to using the Gauss–Newton method to solve the nonlinear, least-squares minimization problem. As a result, the PG-PROM method on which GNAT is based is commonly referred to as the least-squares Petrov–Galerkin (LSPG) projection method in the literature [7]. Essentially, when the Jacobian ($\mathbf{J}$) associated with the Gauss–Newton method results in a symmetric positive definite (SPD) operator, the Galerkin PROM can be shown to minimize the discrete, nonlinear residual in a $\mathbf{J}^{-1}$-norm, whereas the LSPG-PROM can be used for the general case where the Jacobian associated with the Gauss–Newton method is not SPD. For example, when the Jacobian comes from the discretization of the Navier–Stokes equations, the Galerkin PROM approach lacks the minimum-residual optimality property. However, all approximate-then-project methods, such as GNAT, are based on sub-optimal greedy algorithms that take the size of the reduced mesh as input, which is unknown a priori. Recently, a project-then-approximate method for the hyper-reduction of PG-PROMs was developed, presenting an Energy-Conserving Sampling and Weighting (ECSW) type method [13]. This method was extended to PROMs based on local, piecewise-affine approximation subspaces and generalized to finite elements spatial discretizations, as well as finite volume, and finite differences semi-discretizations.

However, in order to evaluate the hyper-reduced residual vectors and Jacobian matrices associated with the selected elements and their neighbors, PG-PROMs require a complementary mesh to be constructed [20]. This means that the hyper-reduced sample mesh must include the patch of elements that contains the selected elements as a complementary mesh, significantly increasing the number of elements required to integrate the hyper-reduced model.[1] The issue arises from the iteration-dependent left ROB of the LSPG-PROM.

In this paper, we propose a novel approach to address the problem of using a complementary mesh for hyper-reduction by introducing an equivalent invariant left ROB $\boldsymbol{\Psi}$ that preserves the 2-norm minimum-residual optimality. We obtain two different invariant ROBs ($\boldsymbol{\Psi} \neq \boldsymbol{\Phi}$) and an alternative PG-PROM by compressing either the converged projected Jacobians onto the right ROB or the non-converged residuals. Additionally, we utilize the empirical cubature method (ECM) to carry out the PG-PROM hyper-reduction without the need for a complementary mesh. ECM identifies a reduced subset of elements and corresponding positive weights, which are calculated from a minimization of the entire unassembled residual projected onto the left ROB $\boldsymbol{\Psi}$. Our proposed approach offers a novel solution to the problem of avoiding the use of a complementary mesh for hyper-reduction and represents a significant contribution to the field.

2. Parametrized problem

Let us consider a general nonlinear parametrized discrete operator of the form

$$\mathbf{R}_t(\mathbf{u}_t; \boldsymbol{\mu}) = \mathbf{R}(\mathbf{u}_{\text{ref}} + \boldsymbol{\Delta}\mathbf{u}(t; \boldsymbol{\mu}), t; \boldsymbol{\mu}) = \mathbf{0}, \tag{1}$$

where $t \in [0, T]$ represents time up to a final time T. With a minor abuse of notation, we use the subscript t to denote that a variable is evaluated at a specific time instance t. The term $\mathbf{u}_t = \mathbf{u}_{\text{ref}} + \boldsymbol{\Delta}\mathbf{u}_t \in \mathbb{R}^N$ designates the nodal state variable evaluated at the specific time instance t. Here, $\mathbf{u}_{\text{ref}}$ represents a reference solution, often based

[1] This behavior should not be confused with hyper-reducing a finite volume model [12,21,22], where it is shown that for the collocation mesh, it is necessary to include neighboring cells (and neighbors of neighbors for second order schemes) to compute the governing equations.

on the solution at the previous time step, $\mathbf{u}_{t-1}$.[2] We have intentionally omitted the explicit dependency of $\mathbf{u}_t$ on the parameter $\boldsymbol{\mu}$ for simplicity of exposition. However, it is important to emphasize that $\mathbf{u}_t$ intrinsically depends on $\boldsymbol{\mu}$, and this dependency will be explicitly denoted when necessary for clarity and instructional purposes. The vector $\boldsymbol{\mu} \in \mathcal{D}$ denotes a specific instance of the input parameters, where $\mathcal{D} \subset \mathbb{R}^P$ represents the discrete point-set of all possible input parameter vectors. The discrete operator $\mathbf{R}_t : \mathbb{R}^N \times \mathcal{D} \to \mathbb{R}^N$ will be referred to as the residual vector. For simplicity of exposition, homogeneous Dirichlet boundary conditions are considered. This high fidelity model will be referred to as the full-order model (FOM). The residual vector is assembled in the usual manner by summing the elemental contributions, which can be expressed as:

$$\mathbf{R}_t = \sum_{e=1}^{L} \mathbf{L}^{e^T} \mathbf{R}_t^e, \tag{2}$$

where L denotes the total number of elements, and $\mathbf{L}^e$ is the assembly operator that connects the degrees of freedom (DOFs) of element e with the global DOFs. The vector $\mathbf{R}_t^e$ represents the values of the elemental residual associated with the DOFs of element e.

An iterative scheme, such as the Newton–Raphson strategy, is required to find a solution $\mathbf{u}_t$ for the nonlinear residual $\mathbf{R}_t$, resulting in the following iterations: for $k = 1, \ldots, K$, solve

$$\boxed{\begin{aligned} \mathbf{J}_t^{(k)}(\mathbf{u}_t^{(k)}; \boldsymbol{\mu})\mathbf{p}_t^{(k)} &= -\mathbf{R}_t^{(k)}(\mathbf{u}_t^{(k)}; \boldsymbol{\mu}) \\ \boldsymbol{\Delta}\mathbf{u}_t^{(k+1)} &= \boldsymbol{\Delta}\mathbf{u}_t^{(k)} + \alpha_t^{(k)}\mathbf{p}_t^{(k)} \\ \mathbf{u}_t^{(k+1)} &= \mathbf{u}_t^{(k)} + \boldsymbol{\Delta}\mathbf{u}_t^{(k+1)}, \end{aligned}} \tag{3}$$

where $\mathbf{J}$ represents the Jacobian operator, and $\boldsymbol{\Delta}\mathbf{u}$ is the increment of the solution state variable at each iteration. $\mathbf{R}_t^{(k)}$ and $\mathbf{J}_t^{(k)}$ are the residual and Jacobian evaluated at the current iterative solution $\mathbf{u}_t^{(k)}$. Specifically, $\mathbf{R}_t^{(k)} = \mathbf{R}_t|_{\mathbf{u}_t^{(k)}} = \mathbf{R}(\mathbf{u}_{\text{ref}} + \boldsymbol{\Delta}\mathbf{u}_t^{(k)}, t; \boldsymbol{\mu})$, and $\mathbf{J}_t^{(k)} = \frac{\partial \mathbf{R}_t}{\partial \mathbf{u}}\big|_{\mathbf{u}_t^{(k)}} = \mathbf{J}(\mathbf{u}_{\text{ref}} + \boldsymbol{\Delta}\mathbf{u}_t^{(k)}, t; \boldsymbol{\mu})$.

The process carries out iterations until convergence is reached, thereby finding the solution $\mathbf{u}_t$ for the specified time step t and the given parameter set $\boldsymbol{\mu}$. Notably, a line search could potentially be employed in the direction of $\mathbf{p} \in \mathbb{R}^N$ to determine the step length α. For a comprehensive understanding and more specific details about the procedure, please refer to Algorithm 1. We used the Newton–Raphson method throughout this work for clarity of presentation. However, it is worth noting that if the Jacobian matrix is not available or computationally expensive to compute, one could use a fixed-point iteration method such as Picard's method instead of the Newton–Raphson method.

Algorithm 1: Newton-Raphson strategy for FOM.

Input: State variable $\mathbf{u}_{\text{ref}}$
Output: State variable $\mathbf{u}_t$ and increment update $\boldsymbol{\Delta}\mathbf{u}_t$
Initialize $\boldsymbol{\Delta}\mathbf{u}_t^{(0)} = \mathbf{0}$, i.e., $\mathbf{u}_t^{(0)} = \mathbf{u}_{\text{ref}}$
for $k = 0, \ldots, K$ (convergence) **do**
  Evaluate residual $\mathbf{R}_t^{(k)} = \mathbf{R}_t|_{\mathbf{u}_t^{(k)}}$
  Evaluate Jacobian $\mathbf{J}_t^{(k)} = \frac{\partial \mathbf{R}_t}{\partial \mathbf{u}}\big|_{\mathbf{u}_t^{(k)}}$
  Solve $\mathbf{J}_t^{(k)}\mathbf{p}_t^{(k)} = -\mathbf{R}_t^{(k)}$
  Update $\boldsymbol{\Delta}\mathbf{u}_t^{(k+1)} = \boldsymbol{\Delta}\mathbf{u}_t^{(k)} + \alpha_t^{(k)}\mathbf{p}_t^{(k)}$
  Update $\mathbf{u}_t^{(k+1)} = \mathbf{u}_t^{(k)} + \boldsymbol{\Delta}\mathbf{u}_t^{(k+1)}$
end for
$\boldsymbol{\Delta}\mathbf{u}_t = \sum_{k=1}^{K} \boldsymbol{\Delta}\mathbf{u}_t^{k+1}$
$\mathbf{u}_t = \mathbf{u}_t^{(K+1)} \equiv \mathbf{u}_{\text{ref}} + \boldsymbol{\Delta}\mathbf{u}_t$

[2] If necessary, the reference solution $\mathbf{u}_{\text{ref}}$ can be enhanced with methods like extrapolation based on the derivatives of $\mathbf{u}_{t-1}$ with respect to time. This can provide more accurate initial conditions, aiding in the stability and convergence of the iterative scheme.

3. Petrov–Galerkin projection

We begin by approximating the solution state variable as a linear combination of n orthogonal basis vectors $\phi_i \in \mathbb{R}^N$ and coefficients $\boldsymbol{\Delta}\hat{\mathbf{u}} \in \mathbb{R}^n$. Here, variables denoted with a hat represent those of the reduced order dimension n. Consequently, we obtain

$$\boldsymbol{\Delta}\tilde{\mathbf{u}}_t(\boldsymbol{\mu}) = \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t(\boldsymbol{\mu}). \tag{4}$$

This approximation enables us to update the solution, now referred to as $\tilde{\mathbf{u}}_t$. The tilde notation indicates the approximate nature of this variable. Thus, the updated solution is given by

$$\tilde{\mathbf{u}}_t = \mathbf{u}_{\text{ref}} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t(\boldsymbol{\mu}). \tag{5}$$

The right ROB $\boldsymbol{\Phi}$ is selected through Proper Orthogonal Decomposition (POD), which involves compressing a set of snapshots collected from the Full Order Model (FOM) simulations for carefully chosen parameters. Specifically, the finite element equations are solved for appropriately chosen values of the input parameter space $\mathcal{D}$, and the state variables are stored in snapshot matrices[3]

$$\mathbf{A}^u = \big[\mathbf{u}_1(\boldsymbol{\mu}_1), \quad \mathbf{u}_2(\boldsymbol{\mu}_1), \quad \ldots, \quad \mathbf{u}_T(\boldsymbol{\mu}_1), \quad \mathbf{u}_1(\boldsymbol{\mu}_2), \quad \mathbf{u}_2(\boldsymbol{\mu}_2), \quad \ldots, \quad \mathbf{u}_T(\boldsymbol{\mu}_2), \quad \ldots, \quad \mathbf{u}_T(\boldsymbol{\mu}_P).\big] \tag{6}$$

The solution basis matrix $\boldsymbol{\Phi}$ is obtained as a linear combination of the columns of $\mathbf{A}^u$, i.e.,

$$col(\boldsymbol{\Phi}) \subset col(\mathbf{A}^u) \tag{7}$$

(the column space of $\boldsymbol{\Phi}$ is a subspace of the column space of $\mathbf{A}^u$). The Proper Orthogonal Decomposition (POD) method aims to reduce the number of columns in $\boldsymbol{\Phi}$ while preserving the essential patterns of the solution state variable. To achieve this, an error threshold $0 \leq \epsilon_u < 1$ is defined, and a basis matrix $\boldsymbol{\Phi}$ is sought such that:

$$\left\|\mathbf{A}^u - \boldsymbol{\Phi}\boldsymbol{\Phi}^T\mathbf{A}^u\right\|_F \leq \epsilon_u \left\|\mathbf{A}^u\right\|_F, \tag{8}$$

where $\|\cdot\|_F$ denotes the Frobenius norm. One approach to finding this matrix is through truncated Singular Value Decomposition (SVD) [23] (see Appendix), which decomposes $\mathbf{A}^u$ as follows:

$$\mathbf{A}^u = \boldsymbol{\Phi}_n\boldsymbol{\Sigma}_n\mathbf{V}_n^T + \mathbf{E}, \tag{9}$$

where $\boldsymbol{\Phi}_n$ is a matrix containing the first n left singular vectors of $\mathbf{A}^u$, $\boldsymbol{\Sigma}_n$ is a diagonal matrix containing the first n singular values, and $\mathbf{V}_n$ is the matrix of the first n right singular vectors. The truncation to n modes allows for a low-rank approximation of $\mathbf{A}^u$, and the remaining term $\mathbf{E}$ accounts for the approximation error.

When Eq. (5) is substituted into Eq. (1), an over-determined system of N nonlinear equations with n unknowns is obtained, where $n \ll N$, given by:

$$\mathbf{R}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) = \mathbf{R}(\mathbf{u}_{\text{ref}} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t(\boldsymbol{\mu}), t; \boldsymbol{\mu}) = \mathbf{0}. \tag{10}$$

To obtain an approximate solution to this system of N nonlinear equations with n unknowns, m constraints are imposed by requiring the orthogonality of the nonlinear residual to a left ROB $\boldsymbol{\Psi} \in \mathbb{R}^{N\times m}$. This process reduces the number of equations in the system from N to m, resulting in a simplified system of m state equations with n unknowns given by

$$\boldsymbol{\Psi}_t^T\mathbf{R}_t(\mathbf{u}_{\text{ref}} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t(\boldsymbol{\mu}); \boldsymbol{\mu}) = \mathbf{0}. \tag{11}$$

Typically, the order of m is similar to the order of n ($\mathcal{O}(m) = \mathcal{O}(n)$), where $m \ll N$. However, to avoid an under-determined system, it is necessary to ensure that m is greater than or equal to n ($m \geq n$) when reducing an over-determined system to a simplified system of m state equations with n unknowns, in order to ensure a well-posed problem. When $m \neq n$, the minimization problem can be addressed using QR decomposition.

[3] Although using $\mathbf{u}$ and $\boldsymbol{\Delta}\mathbf{u}$ in snapshot generation theoretically results in the same column space under exact arithmetic or machine tolerance in randomized singular value decomposition (i.e., $col(\mathbf{A}^u) \subseteq col(\mathbf{A}^{\Delta u})$), we have presented the total approach in this work for its didactic value.

Applying Newton–Raphson to solve the reduced non-linear system to find a solution $\mathbf{u}_t$ results in the following iterations: for $k = 1, \ldots, K$, solve

$$\begin{aligned} \boldsymbol{\Psi}_t^{T(k)} \mathbf{J}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)}; \boldsymbol{\mu}) \boldsymbol{\Phi} \hat{\mathbf{p}}_t^{(k)} &= -\boldsymbol{\Psi}_t^{T(k)} \mathbf{R}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)}; \boldsymbol{\mu}) \\ \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)} &= \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k)} + \alpha_t^{(k)} \hat{\mathbf{p}}_t^{(k)} \\ \tilde{\mathbf{u}}_t^{(k+1)} &= \tilde{\mathbf{u}}_t^{(k)} + \boldsymbol{\Phi} \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)}, \end{aligned} \tag{12}$$

where $\hat{\mathbf{p}}_t^{(k)} \in \mathbb{R}^n$. The detailed procedure is given in Algorithm 2.

The quality of the approximate solution obtained is contingent on both the selection of the basis $\boldsymbol{\Psi}_t$ and the number of constraints imposed. A suitable basis, along with the optimal number of constraints, may be chosen to obtain a sufficiently accurate approximation of the original over-determined system of equations. Although other techniques based on Petrov–Galerkin frameworks have been introduced in the literature where the left ROB changes over iterations, i.e., $\boldsymbol{\Psi}_t^{(k)}$, this work presents a novel approach in Section 5 for a Petrov–Galerkin framework that finds an invariant or constant left ROB, i.e. $\boldsymbol{\Psi}$, while preserving the minimum-residual optimality that will be discussed in the following section.

Algorithm 2: Newton-Raphson strategy for general PG-PROM.

Input: State variable $\mathbf{u}_{\text{ref}}$ and right ROB $\boldsymbol{\Phi}$
Output: State variable $\tilde{\mathbf{u}}_t$ and increment update $\boldsymbol{\Delta}\tilde{\mathbf{u}}_t$

Initialize $\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(0)} = \mathbf{0}$, i.e., $\tilde{\mathbf{u}}_t^{(0)} = \mathbf{u}_{\text{ref}}$
for $k = 0, \ldots, K$ (convergence) **do**
  Evaluate residual $\mathbf{R}_t^{(k)} = \mathbf{R}_t|_{\tilde{\mathbf{u}}_t^{(k)}}$
  Evaluate Jacobian $\mathbf{J}_t^{(k)} = \frac{\partial \mathbf{R}_t}{\partial \tilde{\mathbf{u}}}\big|_{\tilde{\mathbf{u}}_t^{(k)}}$
  Compute $\boldsymbol{\Psi}_t^{T(k)}$ (see Section 3.1)
  Compute $\mathbf{W}^{(k)} = \boldsymbol{\Psi}_t^{T(k)} \mathbf{J}_t^{(k)} \boldsymbol{\Phi}$
  if $m \neq n$ **then**
    Compute QR decomposition $\mathbf{W}^{(k)} = \mathbf{Q}^{(k)} \mathbf{D}^{(k)}$
    Solve $\mathbf{D}^{(k)} \hat{\mathbf{p}}_t^{(k)} = -\mathbf{Q}^{T(k)} \boldsymbol{\Psi}_t^{T(k)} \mathbf{R}_t^{(k)}$
  else
    Solve $\mathbf{W}^{(k)} \hat{\mathbf{p}}_t^{(k)} = -\boldsymbol{\Psi}_t^{T(k)} \mathbf{R}_t^{(k)}$
  end if
  Update $\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)} = \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k)} + \alpha_t^{(k)} \hat{\mathbf{p}}_t^{(k)}$
  Update $\tilde{\mathbf{u}}_t^{(k+1)} = \tilde{\mathbf{u}}_t^{(k)} + \boldsymbol{\Phi} \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)}$
end for
$\boldsymbol{\Delta}\tilde{\mathbf{u}}_t = \boldsymbol{\Phi} \sum_{k=1}^{K} \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{k+1}$
$\tilde{\mathbf{u}}_t = \tilde{\mathbf{u}}_t^{(K+1)} \equiv \mathbf{u}_{\text{ref}} + \boldsymbol{\Delta}\tilde{\mathbf{u}}_t$

3.1. Optimal left ROB

The left ROB $\boldsymbol{\Psi}_t$ is chosen here to minimize the non-linear residual in some norm $\mathbf{G}_t$ in order to meet the minimum-residual optimality of the projection approximation. Specifically, we seek to solve the following minimization problem:

$$\min_{\boldsymbol{\Delta}\hat{\mathbf{u}}_t \in \mathbb{R}^n} \frac{1}{2} \left\| \mathbf{R}_t(\mathbf{u}_{\text{ref}} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t(\boldsymbol{\mu}); \boldsymbol{\mu}) \right\|_{\mathbf{G}_t}^2 \tag{13}$$

where $\|\cdot\|_{\mathbf{G}_t}$ denotes a norm defined by a symmetric, positive definite (SPD) matrix $\mathbf{G}_t \in \mathbb{R}^{N \times N}$. Alternatively, we can write the above problem as follows:

$$\min_{\boldsymbol{\Delta}\hat{\mathbf{u}}_t \in \mathbb{R}^n} \frac{1}{2} \mathbf{R}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu})^T \mathbf{G}_t \mathbf{R}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}). \tag{14}$$

By satisfying the minimum-residual optimality of Eq. (13), a general relationship between projection-based reduced-order models and minimum-residual reduced-order models can be established, as follows:

$$\begin{aligned}\left[\frac{\partial \mathbf{R}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})}{\partial \tilde{\mathbf{u}}}\frac{\partial \tilde{\mathbf{u}}}{\partial \boldsymbol{\Delta}\hat{\mathbf{u}}_t}\right]^T \mathbf{G}_t\mathbf{R}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu}) &= \mathbf{0}\\ \left[\mathbf{J}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})\boldsymbol{\Phi}\right]^T \mathbf{G}_t\mathbf{R}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu}) &= \mathbf{0},\end{aligned} \tag{15}$$

with the structure of the PROM in Eq. (11); this yields

$$\boldsymbol{\Psi}_t = \mathbf{G}_t\mathbf{J}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})\boldsymbol{\Phi}, \tag{16}$$

which is sufficient to satisfy the minimum-residual property for a general PROM. For further details, please refer to [24].

3.1.1. Galerkin projection

The Galerkin projection produces search directions $\hat{\mathbf{p}}_t^{(k)}$ that are minimum-residual optimal with $\mathbf{G}_t = \mathbf{J}_t^{-1}$ when the Jacobians are SPD matrices; that is,

$$\boldsymbol{\Psi}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu}) = \mathbf{J}_t^{-1}(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})\mathbf{J}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})\boldsymbol{\Phi} = \boldsymbol{\Phi}, \tag{17}$$

where the right ROB $\boldsymbol{\Phi}$ is constant. Consequently, the left ROB no longer depends on time and the parameter, becoming $\boldsymbol{\Psi}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu}) = \boldsymbol{\Psi}$, see Ref. [25]. The Newton–Raphson system is then transformed into the following iterations: for $k = 1, \ldots, K$, solve

$$\boxed{\begin{aligned}\boldsymbol{\Phi}^T\mathbf{J}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)};\boldsymbol{\mu})\boldsymbol{\Phi}\hat{\mathbf{p}}_t^{(k)} &= -\boldsymbol{\Phi}^T\mathbf{R}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)};\boldsymbol{\mu})\\ \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)} &= \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k)} + \alpha_t^{(k)}\hat{\mathbf{p}}_t^{(k)}\\ \tilde{\mathbf{u}}_t^{(k+1)} &= \tilde{\mathbf{u}}_t^{(k)} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)},\end{aligned}} \tag{18}$$

see Algorithm 3 for more details.

However, the Galerkin projection does not guarantee that the residual is minimized since the Jacobians of a nonlinear problem are not in general SPD matrices. As a result, a different projection for general non-linear problems was introduced in Ref. [7], and is discussed in the following section.

Algorithm 3: Newton-Raphson strategy for Galerkin-PROM.

Input: State variable $\mathbf{u}_{\text{ref}}$ and right ROB $\boldsymbol{\Phi}$
Output: State variable $\tilde{\mathbf{u}}_t$ and increment update $\boldsymbol{\Delta}\tilde{\mathbf{u}}_t$

Initialize $\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(0)} = \mathbf{0}$, i.e., $\tilde{\mathbf{u}}_t^{(0)} = \mathbf{u}_{\text{ref}}$
for $k = 0, \ldots, K$ (convergence) **do**
  Evaluate residual $\mathbf{R}_t^{(k)} = \mathbf{R}_t|_{\tilde{\mathbf{u}}_t^{(k)}}$
  Evaluate Jacobian $\mathbf{J}_t^{(k)} = \frac{\partial \mathbf{R}_t}{\partial \tilde{\mathbf{u}}}\big|_{\tilde{\mathbf{u}}_t^{(k)}}$
  Compute $\mathbf{W}^{(k)} = \boldsymbol{\Phi}^T\mathbf{J}_t^{(k)}\boldsymbol{\Phi}$
  Solve $\mathbf{W}^{(k)}\hat{\mathbf{p}}_t^{(k)} = -\boldsymbol{\Phi}^T\mathbf{R}_t^{(k)}$
  Update $\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)} = \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k)} + \alpha_t^{(k)}\hat{\mathbf{p}}_t^{(k)}$
  Update $\tilde{\mathbf{u}}_t^{(k+1)} = \tilde{\mathbf{u}}_t^{(k)} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)}$
end for
$\boldsymbol{\Delta}\tilde{\mathbf{u}}_t = \boldsymbol{\Phi}\sum_{k=1}^{K}\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{k+1}$
$\tilde{\mathbf{u}}_t = \tilde{\mathbf{u}}_t^{(K+1)} \equiv \mathbf{u}_{\text{ref}} + \boldsymbol{\Delta}\tilde{\mathbf{u}}_t$

3.1.2. Least-squares Petrov–Galerkin (LSPG) projection

Nonlinear problems often result in Jacobians that are not SPD. In such cases, the LSPG method satisfies the condition in Eq. (16) by setting $\mathbf{G}_t = \mathbf{I}$, where $\mathbf{I} \in \mathbb{R}^{N\times N}$ is the identity matrix. This ensures that the method exhibits the minimum-residual property, leading to the following equation:

$$\boldsymbol{\Psi}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu}) = \mathbf{J}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})\boldsymbol{\Phi}. \tag{19}$$

The above equation leads to the Gauss–Newton iterations, which are given by the following steps: for $k = 1, \ldots, K$, solve

$$\begin{aligned}\underbrace{\left[\mathbf{J}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)};\boldsymbol{\mu})\boldsymbol{\Phi}\right]^T}_{\boldsymbol{\Psi}_t^{T(k)}(\tilde{\mathbf{u}}_t^{(k)};\boldsymbol{\mu})}\mathbf{J}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)};\boldsymbol{\mu})\boldsymbol{\Phi}\hat{\mathbf{p}}_t^{(k)} &= -\underbrace{\left[\mathbf{J}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)};\boldsymbol{\mu})\boldsymbol{\Phi}\right]^T}_{\boldsymbol{\Psi}_t^{T(k)}(\tilde{\mathbf{u}}_t^{(k)};\boldsymbol{\mu})}\mathbf{R}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)};\boldsymbol{\mu})\\ \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)} &= \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k)} + \alpha_t^{(k)}\hat{\mathbf{p}}_t^{(k)}\\ \tilde{\mathbf{u}}_t^{(k+1)} &= \tilde{\mathbf{u}}_t^{(k)} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)}.\end{aligned} \tag{20}$$

Note that the chosen left ROB $\boldsymbol{\Psi}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu}) = \mathbf{J}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})\boldsymbol{\Phi}$ changes from one Newton iteration to another (see Algorithm 4 for more details).

Algorithm 4: Gauss-Newton strategy for LSPG-PROM.

Input: State variable $\mathbf{u}_{\text{ref}}$ and right ROB $\boldsymbol{\Phi}$
Output: State variable $\tilde{\mathbf{u}}_t$ and increment update $\boldsymbol{\Delta}\tilde{\mathbf{u}}_t$
Initialize $\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(0)} = \mathbf{0}$, i.e., $\tilde{\mathbf{u}}_t^{(0)} = \mathbf{u}_{\text{ref}}$
for $k = 0, \ldots, K$ (convergence) **do**
  Evaluate residual $\mathbf{R}_t^{(k)} = \mathbf{R}_t|_{\tilde{\mathbf{u}}_t^{(k)}}$
  Evaluate Jacobian $\mathbf{J}_t^{(k)} = \frac{\partial \mathbf{R}_t}{\partial \tilde{\mathbf{u}}}\big|_{\tilde{\mathbf{u}}_t^{(k)}}$
  Compute left ROB $\boldsymbol{\Psi}_t^{T(k)} = \left[\mathbf{J}_t^{(k)}\boldsymbol{\Phi}\right]^T$
  Compute $\mathbf{W}^{(k)} = \boldsymbol{\Psi}_t^{T(k)}\mathbf{J}_t^{(k)}\boldsymbol{\Phi}$
  Solve $\mathbf{W}^{(k)}\hat{\mathbf{p}}_t^{(k)} = -\boldsymbol{\Psi}_t^{T(k)}\mathbf{R}_t^{(k)}$
  Update $\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)} = \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k)} + \alpha_t^{(k)}\hat{\mathbf{p}}_t^{(k)}$
  Update $\tilde{\mathbf{u}}_t^{(k+1)} = \tilde{\mathbf{u}}_t^{(k)} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)}$
end for
$\boldsymbol{\Delta}\tilde{\mathbf{u}}_t = \boldsymbol{\Phi}\sum_{k=1}^{K}\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{k+1}$
$\tilde{\mathbf{u}}_t = \tilde{\mathbf{u}}_t^{(K+1)} \equiv \mathbf{u}_{\text{ref}} + \boldsymbol{\Delta}\tilde{\mathbf{u}}_t$

Nonlinear PROMs typically involve at least one step that increases in computational complexity with the size of the FOM, even though the nonlinear system that defines the reduced-order model is much smaller. To overcome this bottleneck, many nonlinear PROM methods include a hyper-reduction technique that adds an additional approximation. For instance, in Newton–Raphson methods, the residual vector $\mathbf{R}_t^{(k)}(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})$, Jacobian matrix $\mathbf{J}_t^{(k)}(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})$, and matrix–vector products for all entities or elements of the FOM are evaluated at each iteration for all time steps. Hyper-reduction techniques can tackle this bottleneck by evaluating these vectors and matrices over a subset of elements, significantly reducing the computational burden without compromising accuracy. It is worth noting that if the left ROB is variant or iteration-dependent ($\boldsymbol{\Psi}_t^{(k)}(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})$), it may pose a disadvantage to the "hyper-reduction" strategy. In such cases, constructing the PROM element by element becomes infeasible, and a complementary mesh needs to be used for the reduced mesh. Details regarding the need for a complementary mesh approach are discussed in the following section.

4. Hyper-reduction

To discuss hyper-reduction, let us first revisit Eq. (11) for convenience:

$$\boldsymbol{\Psi}_t^T\mathbf{R}_t(\mathbf{u}_{\text{ref}} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t(\boldsymbol{\mu});\boldsymbol{\mu}) = \mathbf{0}.$$

For the assembly of elemental contributions, this expression takes the following form:

$$\boldsymbol{\Psi}_t^T\mathbf{R}_t = \boldsymbol{\Psi}_t^T\sum_{e=1}^{L}\mathbf{L}^{e^T}\mathbf{R}_t^e = \sum_{e=1}^{L}(\mathbf{L}^e\boldsymbol{\Psi}_t)^T\mathbf{R}_t^e = \sum_{e=1}^{L}\boldsymbol{\Psi}_t^{e^T}\mathbf{R}_t^e = \mathbf{0}, \tag{21}$$

where L denotes the total number of elements, and for each element, $\mathbf{L}^e$ is the assembly operator, $\mathbf{R}_t^e$ is the elemental residual vector, and $\boldsymbol{\Psi}_t^e$ represents the values of the left ROB associated with the elemental degrees of freedom. As

previously mentioned, this equation shows that although the equation-solving effort has drastically diminished with $n \ll N$, the computational complexity of the problem still scales with the size of the discretization of the underlying finite elements. This is because it is necessary to loop over the L finite elements of the mesh to assemble the residual vectors and Jacobian matrices.

As demonstrated in [13,15], one may approximate the Petrov–Galerkin projection of the residual onto the left ROB, given by Eq. (21), by finding a subset of elements $\mathbf{z}$ and positive weights $\boldsymbol{\omega}$ such that

$$\sum_{e\in\mathbf{z}} \omega^e \boldsymbol{\Psi}_t^{e^T} \mathbf{R}_t^e = \mathbf{0}. \tag{22}$$

In other words, the subset of elements and their weights are chosen such that the resulting approximation of the residual vector satisfies the projection property of the reduced order model. Specifically, the approximation of the residual vector is obtained by summing over the chosen elements in $\mathbf{z}$, where for each element e in $\mathbf{z}$, the corresponding weight $\omega^e \in \boldsymbol{\omega}$ is multiplied by the residual vector $\mathbf{R}_t^e$ and its corresponding left ROB values $\boldsymbol{\Psi}_t^{e^T}$.

The optimization problem for determining $\mathbf{z}$ and $\boldsymbol{\omega}$ involves solving Eq. (21) using the same input parameters as in the first reduction stage. The reduced equation is expressed in matrix format as follows:

$$\sum_{e=1}^{L} \boldsymbol{\Psi}_i^{e^T} \mathbf{R}_i^e(\boldsymbol{\mu}_j) = \left[\boldsymbol{\Psi}^{1^T}\mathbf{R}_i^1(\boldsymbol{\mu}_j), \quad \boldsymbol{\Psi}^{2^T}\mathbf{R}_i^2(\boldsymbol{\mu}_j), \quad \ldots, \quad \boldsymbol{\Psi}^{L^T}\mathbf{R}_i^L(\boldsymbol{\mu}_j)\right]\mathbb{1} = \mathbf{b}_i(\boldsymbol{\mu}_j), \qquad i = 1, 2, \ldots, T, \tag{23}$$

where $\mathbf{b}_i(\boldsymbol{\mu}_j) \in \mathbb{R}^m$ represents the sum of the projected residuals for all elements at the ith time step for the jth parameter instance. Here, $\mathbb{1}$ denotes an all-ones vector.

Remark 4.1. Using the all-ones vector assumes that the mesh elements are similar enough to each other, such that each element's contribution to the projected residual is approximately equal. An alternative approach is to create a weighting vector that incorporates a coefficient to account for the varying contributions, such as the volume of each element [14]. If a weighting vector is used, then the residual should be scaled component-wise by its corresponding weight.

Let us construct a matrix $\mathbf{X} \in \mathbb{R}^{(m\times T\times P)\times L}$ by collecting the block column matrix from Eq. (23) for all training parameters, which is given by:

$$\mathbf{X} = \left[\mathbf{X}_1(\boldsymbol{\mu}_1), \quad \mathbf{X}_2(\boldsymbol{\mu}_1), \quad \ldots \quad \mathbf{X}_T(\boldsymbol{\mu}_1), \quad \mathbf{X}_1(\boldsymbol{\mu}_2), \quad \mathbf{X}_2(\boldsymbol{\mu}_2), \quad \ldots, \quad \mathbf{X}_T(\boldsymbol{\mu}_2), \quad \ldots, \quad \mathbf{X}_T(\boldsymbol{\mu}_P)\right]^T, \tag{24}$$

where each block column $\mathbf{X}_i(\boldsymbol{\mu}_j) \in \mathbb{R}^{L\times m}$ is given by:

$$\mathbf{X}_i(\boldsymbol{\mu}_j) = \begin{bmatrix} (\boldsymbol{\Psi}^{1^T}\mathbf{R}_i^1(\boldsymbol{\mu}_j))^T \\ (\boldsymbol{\Psi}^{2^T}\mathbf{R}_i^2(\boldsymbol{\mu}_j))^T \\ \vdots \\ (\boldsymbol{\Psi}^{L^T}\mathbf{R}_i^L(\boldsymbol{\mu}_j))^T \end{bmatrix}. \tag{25}$$

The discrete minimization problem statement is very similar to the statement for the continuous problem in Ref. [26]. The objective is to determine the optimal subset of elements and their corresponding weights. Therefore, we need to find $\boldsymbol{\omega} \in \mathbb{R}_+^\ell$ and $\mathbf{z} \subset 1, 2, \ldots, L$ such that:

$$(\boldsymbol{\omega}, \mathbf{z}) = \arg\min_{\bar{\boldsymbol{\omega}}\geq\mathbf{0},\bar{\mathbf{z}}} \|\mathbf{X}_{\bar{\mathbf{z}}}\bar{\boldsymbol{\omega}} - \mathbf{b}\|^2, \tag{26}$$

where $\mathbf{X}_\mathbf{z}$ is the block matrix of $\mathbf{X}$ formed by the columns corresponding to the indexes $\mathbf{z}$, and $\boldsymbol{\omega} = [\omega^1, \omega^2, \ldots, \omega^\ell]^T$. The goal is to select a set of ℓ elements from the L elements of the underlying finite element mesh $x_1, x_2, \ldots, x_L$. For more information on the "Empirical Cubature Method" (ECM) for finding the optimal elements and weights, we refer the reader to Refs. [14,15]. Unlike other methods in the literature [13,20], the Empirical Cubature Method solves the minimization problem in Eq. (26) in terms of a set of orthogonal basis vectors rather than snapshots of the raw discrete integrand $\mathbf{X}$. To this end, the ECM requires obtaining an orthonormal matrix via a truncated SVD as

$$\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\boldsymbol{\Theta} + \mathbf{E}. \tag{27}$$

The matrix $\boldsymbol{\Theta} \in \mathbb{R}^{p \times L}$ is used to define a vector $\mathbf{b}_{\boldsymbol{\Theta}} \in \mathbb{R}^{L}$ as

$$\boldsymbol{\Theta}\mathbb{1} = \mathbf{b}_{\boldsymbol{\Theta}}, \tag{28}$$

such that the equivalent optimization problem statement in Eq. (26) becomes:

$$(\boldsymbol{\omega}, \mathbf{z}) = \arg \min_{\tilde{\boldsymbol{\omega}} \geq \mathbf{0}, \bar{\mathbf{z}}} \|\boldsymbol{\Theta}_{\bar{\mathbf{z}}} \tilde{\boldsymbol{\omega}} - b_{\boldsymbol{\Theta}}\|^2. \tag{29}$$

The ECM algorithm for finding the optimal subset of elements and weights is described in detail in [15], and is used in this work to address the hyper-reduction strategy. Note that the hyper-reduction approximation does not aim to accurately represent the full-order residual (FOM), but only its projection onto the left subspace spanned by the columns of the left reduced order basis (ROM).

The hyper-reduction technique has been extensively studied in previous literature for the Galerkin projection [9–11,13–17]. However, since it is a relatively new approach, only a limited number of studies have investigated scenarios where the left reduced-order basis (ROB) differs [7,20,22,27]. This introduces some unique challenges to the hyper-reduction scheme.

4.1. Extension to least-squares problems

Let us consider the case when the minimization is performed in the 2-norm, which corresponds to $\mathbf{G}_t = \mathbf{I}$. In this case, Eq. (13) becomes

$$\boldsymbol{\Phi}^T \mathbf{J}_t^T(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) \mathbf{R}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) = \mathbf{0}. \tag{30}$$

We define the Jacobian-weighted residual vector as follows:

$$\bar{\mathbf{R}}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) = \mathbf{J}_t^T \mathbf{R}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) = \mathbf{0}. \tag{31}$$

With this definition at hand, we can rewrite Eq. (30) as

$$\boldsymbol{\Phi}^T \bar{\mathbf{R}}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) = \mathbf{0}, \tag{32}$$

which, except for the $\bar{\mathbf{R}}_t$ operator, is identical to Eq. (11) in the sense that it involves a ROB-weighted residual quantity. However, because the Jacobian-weighted residual vector involves global quantities, it cannot be assembled locally element by element. This means that it requires information from a "complementary mesh", which is an additional set of elements used to accurately represent the global behavior of the system on the reduced mesh.

To further clarify this point, we can express Eq. (32) in terms of elemental contributions as follows:

$$\begin{aligned} \boldsymbol{\Phi}^T \mathbf{J}_t^T \mathbf{R}_t &= \boldsymbol{\Phi}^T \left(\sum_{e=1}^{L} \mathbf{L}^{e^T} \mathbf{J}_t^{e^T} \mathbf{L}^e \right) \mathbf{R}_t \\ &= (\sum_{e=1}^{L} \left(\mathbf{L}^e \boldsymbol{\Phi} \right)^T \mathbf{J}_t^{e^T} \mathbf{L}^e) \mathbf{R}_t \\ &= \left(\sum_{e=1}^{L} \boldsymbol{\Phi}^{e^T} \mathbf{J}_t^{e^T} \mathbf{L}^e \right) \mathbf{R}_t, \end{aligned} \tag{33}$$

where we introduce the elemental quantity

$$\mathbf{R}_t^{Le} := \mathbf{L}^e \mathbf{R}_t. \tag{34}$$

The quantity $\mathbf{R}_t^{Le}$ signifies the entries of the assembled residual vector that correspond to the degrees of freedom of element e. It is important not to confuse this quantity with $\mathbf{R}^e$, which denotes the contribution of element e to the residual vector. To emphasize this distinction, note that $\mathbf{R}_t^{Le}$ and $\mathbf{R}^e$ are not the same, i.e., $\mathbf{R}_t^{Le} \neq \mathbf{R}^e$.

Therefore, we have

$$\sum_{e=1}^{L} \boldsymbol{\Phi}^{e^T} \bar{\mathbf{R}}_t^e(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) = \mathbf{0}, \tag{35}$$

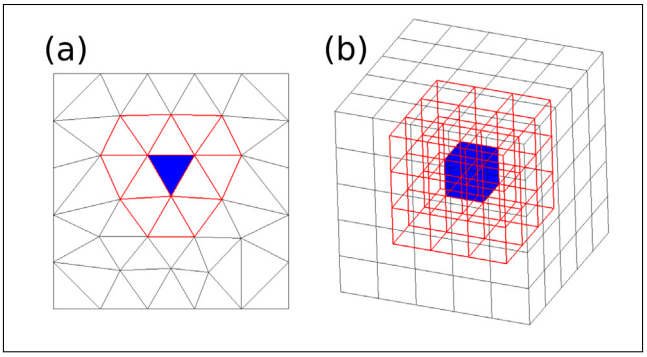


Fig. 1. Patch of elements associated to a selected element of a 2D and 3D mesh. (a) 2D, (b) 3D. Red: Patch of elements, Blue: Selected element. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

where

$$\bar{\mathbf{R}}_t^e := \mathbf{J}_t^{e^T} \mathbf{R}_t^{Le}. \tag{36}$$

The main difference between the LSPG scheme and other schemes, such as Galerkin, is that the computation of $\bar{\mathbf{R}}_t^e$ in the former requires determining residual vectors for all elements sharing nodes with element e, forming a patch of elements, as shown in Fig. 1 . This feature necessitates constructing a complementary mesh based on the patch of elements for a given set of elements $\mathbf{z} \subset 1, 2, \ldots, L$, regardless of the element selection method employed in the hyper-reduction process [20]. This peculiarity of problems arising from minimization of residuals suggests that the ECM search process should be adjusted such that, after selecting a specific element, its associated patches are considered as candidates for the next step in the algorithm. This maximizes the overlap between patches. Using the hyper-reduction scheme described in Section 4 for the variant left ROB would result in an increased number of elements to consider in the finite element analyses.

5. Invariant left ROB approach

5.1. Formulation

Let us revisit Eq. (30), which is reproduced below for convenience (in the case where $\mathbf{G}_t = \mathbf{I}$):

$$\mathbf{r}_t = \mathbf{\Phi}^T \mathbf{J}_t^T(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) \mathbf{R}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) = \mathbf{0}.$$

This nonlinear equation can be interpreted as a Petrov–Galerkin projection in which the left subspace or the subspace of constraints (a term borrowed from [25]) changes at each iteration. The left subspace in the preceding equation is given by $col(\mathbf{J}_t\mathbf{\Phi})$, where $\mathbf{J}_t$ is the Jacobian of the original finite element equations, and this matrix evolves during the iterations. In light of the preceding observation, one may wonder whether it is possible to reformulate Eq. (30) so that the left ROB matrix is held fixed during the iterations.

The problem may be posed as follows: find an orthogonal matrix $\mathbf{\Psi} \in \mathbb{R}^{N \times m}$ such that

$$\mathbf{\Psi}^T \mathbf{R}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) = \mathbf{0} \tag{37}$$

for all input parameters $\boldsymbol{\mu}$ used for determining the right ROB $\mathbf{\Phi} \in \mathbb{R}^{N \times n}$.

In what follows, we describe and discuss two possible approaches for determining the left ROB $\mathbf{\Psi}$. These two alternatives will introduce a second training stage in which data will be gathered from the LSPG-PROM solution, leading to an increase in offline computational cost. However, we demonstrate in the sequel that this increase in computational effort is greatly repaid in the online stage, as the resulting PG-PROM requires far fewer elements than the standard LSPG-PROM.

5.1.1. Jacobian-based approach

Jacobian basis procedure

1. This approach involves solving Eq. 5.1 while collecting the matrices $\mathbf{J}_i(\tilde{\mathbf{u}}_i;\boldsymbol{\mu}_j)\boldsymbol{\Phi}$ when convergence is achieved (for all training parameters $\boldsymbol{\mu}$).
   First, define the auxiliary matrix:
   $$\mathbf{S}^j_{\mathbf{J}}(\boldsymbol{\mu}_j) \coloneqq \left[\mathbf{J}_1(\tilde{\mathbf{u}}_1;\boldsymbol{\mu}_j)\boldsymbol{\Phi},\quad \mathbf{J}_2(\tilde{\mathbf{u}}_2;\boldsymbol{\mu}_j)\boldsymbol{\Phi},\quad \ldots,\quad \mathbf{J}_T(\tilde{\mathbf{u}}_T;\boldsymbol{\mu}_j)\boldsymbol{\Phi}\right]. \tag{38}$$
   Then construct $\mathbf{S}_{\mathbf{J}}$ by stacking these matrices:
   $$\mathbf{S}_{\mathbf{J}} = \left[\mathbf{S}^1_{\mathbf{J}}(\boldsymbol{\mu}_1),\quad \mathbf{S}^2_{\mathbf{J}}(\boldsymbol{\mu}_2),\quad \ldots,\quad \mathbf{S}^P_{\mathbf{J}}(\boldsymbol{\mu}_P)\right]. \tag{39}$$
2. Once the matrix has been constructed, the truncated SVD (with relative truncation tolerance $\epsilon_{\boldsymbol{\Psi}_{\mathbf{J}}}$) is used to determine an orthogonal basis matrix:
   $$\left[\boldsymbol{\Psi}_{\mathbf{J}}, \bullet, \bullet\right] = \mathrm{SVD}(\mathbf{S}_{\mathbf{J}}, \epsilon_{\boldsymbol{\Psi}_{\mathbf{J}}}). \tag{40}$$

It is worth noting that although all iterative schemes in this work are presented using the Newton–Raphson method, the proposed approach is not restricted to this method and can be applied to any iterative scheme, including those that use fixed-point iteration techniques such as Picard's method. Therefore, the matrices $\mathbf{J}_i$ can be computed using either the exact Jacobian or an approximate Jacobian.

Observations:

1. The number of columns of $\boldsymbol{\Psi}_{\mathbf{J}}$ is always equal to or greater than the number of columns of $\boldsymbol{\Phi}$:
   $$ncol(\boldsymbol{\Psi}_{\mathbf{J}}) \geq ncol(\boldsymbol{\Phi}). \tag{41}$$
   The equality holds in the linear case, i.e., when $\mathbf{J}_t$ is constant.
2. As a corollary, in the general case where $ncol(\boldsymbol{\Psi}_{\mathbf{J}}) > ncol(\boldsymbol{\Phi})$, Eq. (37) becomes an over-determined system of nonlinear equations. This implies that the problem of Eq. (37) should be posed as finding $\boldsymbol{\Delta}\hat{\mathbf{u}}_t \in \mathbb{R}^n$ such that
   $$\min_{\boldsymbol{\Delta}\hat{\mathbf{u}}_t\in\mathbb{R}^n} \frac{1}{2}\left\| \boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{R}_t(\tilde{\mathbf{u}}_t;\boldsymbol{\mu})\right\|^2, \tag{42}$$
   which in turn, leads to the stationary condition
   $$\begin{aligned}\frac{\partial \frac{1}{2}(\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{R}_t)^T(\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{R}_t)}{\partial\boldsymbol{\Delta}\hat{\mathbf{u}}_t} &= \frac{1}{2}(\boldsymbol{\Psi}_{\mathbf{J}}^T\frac{\partial\mathbf{R}_t}{\partial\boldsymbol{\Delta}\hat{\mathbf{u}}_t})^T(\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{R}_t) + \frac{1}{2}(\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{R}_t)^T(\boldsymbol{\Psi}_{\mathbf{J}}^T\frac{\partial\mathbf{R}_t}{\partial\boldsymbol{\Delta}\hat{\mathbf{u}}_t})\\ &= \frac{1}{2}(\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{J}_t\boldsymbol{\Phi})^T(\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{R}_t) + \frac{1}{2}(\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{R}_t)^T(\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{J}_t\boldsymbol{\Phi})\\ &= (\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{J}_t\boldsymbol{\Phi})^T(\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{R}_t) = \mathbf{0}.\end{aligned} \tag{43}$$
   Hence, in this case, the residual adopts the expression
   $$\mathbf{r}'_t = \boldsymbol{\Phi}^T(\mathbf{J}_t^T\boldsymbol{\Psi}_{\mathbf{J}})(\boldsymbol{\Psi}_{\mathbf{J}}^T\mathbf{R}_t) = \mathbf{0}. \tag{44}$$

If we use the standard Newton–Raphson method to solve the preceding equation, obtain the following system of linear equations at each iteration:

$$\frac{\partial\mathbf{r}'}{\partial\boldsymbol{\Delta}\hat{\mathbf{u}}_t}\hat{\mathbf{p}}_t = -\mathbf{r}'_t, \tag{45}$$

where the Jacobian matrix of the projected residual $\mathbf{r}'$ is given by

$$\frac{\partial \mathbf{r}'_t}{\partial \boldsymbol{\Delta}\hat{\mathbf{u}}_t} = \boldsymbol{\Phi}^T \frac{\partial \mathbf{J}_t^T}{\partial \boldsymbol{\Delta}\hat{\mathbf{u}}_t} \boldsymbol{\Psi}_{\mathbf{J}} \boldsymbol{\Psi}_{\mathbf{J}}^T \mathbf{R}_t + \boldsymbol{\Phi}^T \mathbf{J}_t^T \boldsymbol{\Psi}_{\mathbf{J}} \boldsymbol{\Psi}_{\mathbf{J}}^T \frac{\partial \mathbf{R}_t}{\partial \boldsymbol{\Delta}\hat{\mathbf{u}}_t} \tag{46}$$

Near the minimum, i.e., when $\mathbf{R}_t \approx \mathbf{0}$, we can discard the first term on the right-hand side of the above expression, leading to the approximation

$$\frac{\partial \mathbf{r}'}{\partial \boldsymbol{\Delta}\hat{\mathbf{u}}_t} \approx \boldsymbol{\Phi}^T \mathbf{J}_t^T \boldsymbol{\Psi}_{\mathbf{J}} \boldsymbol{\Psi}_{\mathbf{J}}^T \mathbf{J}_t \boldsymbol{\Phi}, \tag{47}$$

which results in the following Newton–Raphson iterations: for $k = 1, \ldots, K$, solve

$$\begin{aligned} \left[\mathbf{J}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)}; \boldsymbol{\mu})\boldsymbol{\Phi}\right]^T \boldsymbol{\Psi}_{\mathbf{J}} \boldsymbol{\Psi}_{\mathbf{J}}^T \mathbf{J}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)}; \boldsymbol{\mu})\boldsymbol{\Phi}\hat{\mathbf{p}}_t^{(k)} &= -\left[\mathbf{J}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)}; \boldsymbol{\mu})\boldsymbol{\Phi}\right]^T \boldsymbol{\Psi}_{\mathbf{J}} \boldsymbol{\Psi}_{\mathbf{J}}^T \mathbf{R}_t^{(k)}(\tilde{\mathbf{u}}_t^{(k)}; \boldsymbol{\mu}) \\ \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)} &= \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k)} + \alpha_t^{(k)} \hat{\mathbf{p}}_t^{(k)} \\ \tilde{\mathbf{u}}_t^{(k+1)} &= \tilde{\mathbf{u}}_t^{(k)} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)}. \end{aligned} \tag{48}$$

It should be noted that solving the system of equations above amounts to minimizing the expression

$$\min_{\hat{\mathbf{p}}_t \in \mathbb{R}^n} \left\| \boldsymbol{\Psi}_{\mathbf{J}}^T \mathbf{J}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu})\boldsymbol{\Phi}\hat{\mathbf{p}}_t - \boldsymbol{\Psi}_{\mathbf{J}}^T \mathbf{R}_t(\tilde{\mathbf{u}}_t; \boldsymbol{\mu}) \right\|^2 \tag{49}$$

at each iteration. This minimization problem can be addressed by using the QR factorization of

$$\boldsymbol{\Psi}_{\mathbf{J}}^T \mathbf{J}_t \boldsymbol{\Phi} \in \mathbb{R}^{m \times n}. \tag{50}$$

Note that although this matrix is still rectangular, it has a much smaller dimension ($m \ll N$ and $n \ll N$) compared to the full order model $\in \mathbb{R}^{N \times N}$.

Proposition. *If $\epsilon_{\Psi_{\mathbf{J}}} = \mathbf{0}$ in the SVD of Eq. (40), then Eq. (44) is equivalent to Eq. (37) for the training parameters.*

Proof. Eq. (44) can be written as

$$(\boldsymbol{\Psi}_{\mathbf{J}} \boldsymbol{\Psi}_{\mathbf{J}}^T \mathbf{J}_t \boldsymbol{\Phi})^T \mathbf{R}_t = \mathbf{0}. \tag{51}$$

If $\epsilon_{\Psi_{\mathbf{J}}} = \mathbf{0}$ in Eq. (40), then $\mathbf{J}_t\boldsymbol{\Phi} \in col(\boldsymbol{\Psi}_{\mathbf{J}})$ for the training parameters. This means that, since $\boldsymbol{\Psi}_{\mathbf{J}}$ is orthogonal, $\boldsymbol{\Psi}_{\mathbf{J}} \boldsymbol{\Psi}_{\mathbf{J}}^T \mathbf{J}_t \boldsymbol{\Phi} = \mathbf{J}_t \boldsymbol{\Phi}$. Thus, Eq. (40) becomes

$$(\mathbf{J}_t \boldsymbol{\Phi})^T \mathbf{R}_t = \mathbf{0}. \tag{52}$$

□

As the number of modes increases, such as when solving highly nonlinear problems, the size of the snapshots matrix in Eq. (39) can potentially become quite large. This is because the size of the matrix increases linearly with the number of modes. This observation highlights the need to explore alternative techniques that can better handle large, evolving datasets. One such approach is based on residual evolution, which we will discuss in more detail below.

5.1.2. Residual-based approach

The orthogonal matrix $\boldsymbol{\Psi}$ introduces an orthogonal decomposition of $\mathbb{R}^N$ as shown in Eq. (37):

$$\mathbb{R}^N = null(\boldsymbol{\Psi}^T) \oplus col(\boldsymbol{\Psi}). \tag{53}$$

It is important to note that a converged nodal residual vector will belong to $null(\boldsymbol{\Psi}^T)$, while non-converged nodal residual vectors, will have components in $col(\boldsymbol{\Psi})$. This observation suggests an alternative approach for determining the left ROB as a basis matrix for the column space of non-converged residuals. The detailed procedure for this approach is provided below. See Fig. 2.

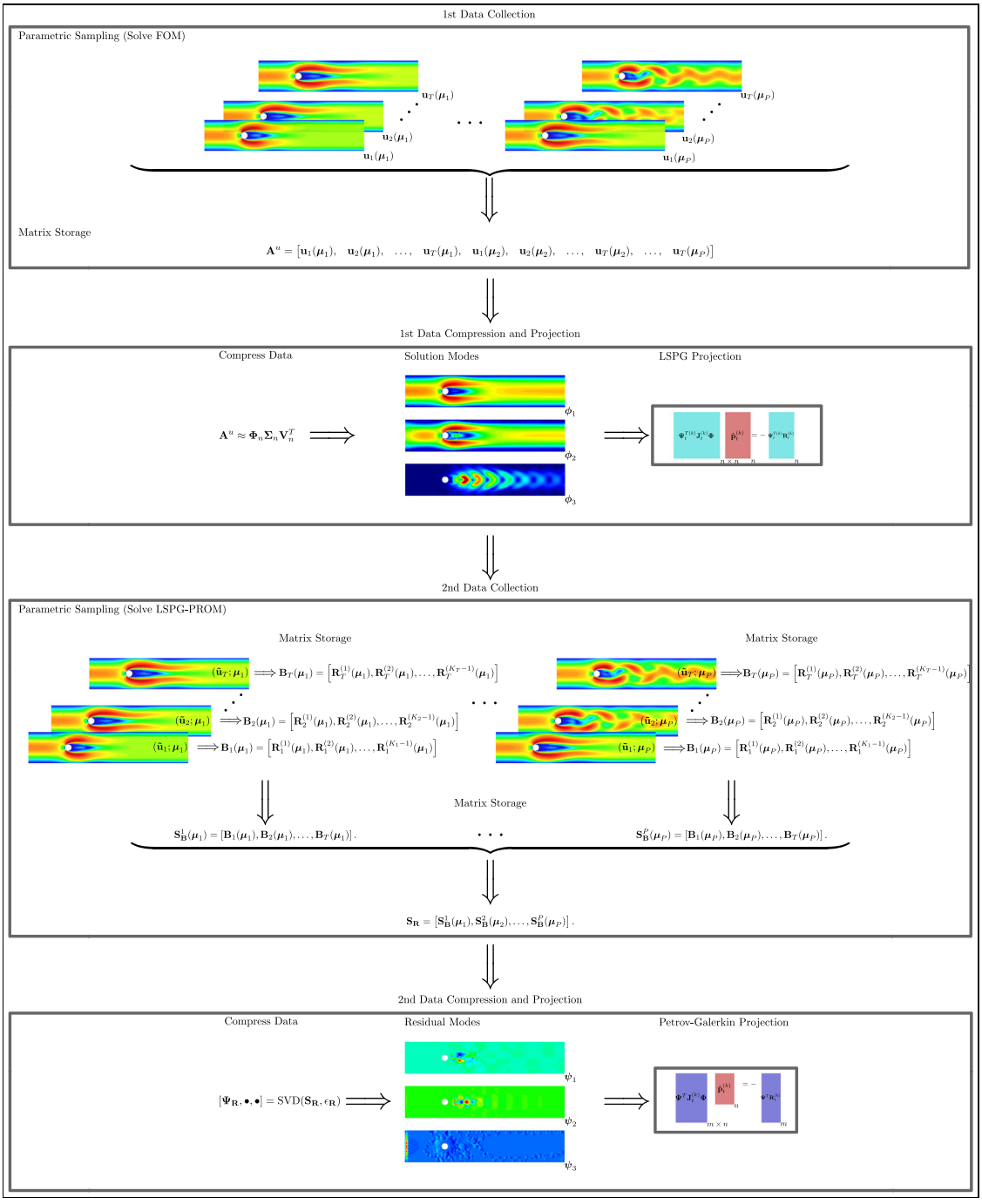


Fig. 2. Schematic overview of Petrov–Galerkin PROM construction.

Residuals basis procedure

1. For each training parameter $\boldsymbol{\mu}_j$ and at each nonlinear iteration k, define the non-converged residual at timestep i as:

$$\mathbf{R}_i^{(k)}(\boldsymbol{\mu}_j) = \mathbf{R}_i(\mathbf{u}_i^{(k)}; \boldsymbol{\mu}_j), \tag{54}$$

   Here, k signifies the kth nonlinear iteration. For every timestep i and each training parameter $\boldsymbol{\mu}_j$, gather a matrix of snapshots of the non-converged nodal residual vectors associated with the solutions $\tilde{\mathbf{u}}_i^{(k)}$:

$$\mathbf{B}_i(\boldsymbol{\mu}_j) = \left[\mathbf{R}_i^{(1)}(\boldsymbol{\mu}_j), \mathbf{R}_i^{(2)}(\boldsymbol{\mu}_j), \ldots, \mathbf{R}_i^{(K_i-1)}(\boldsymbol{\mu}_j)\right], \tag{55}$$

   where K_i denotes the total amount of non-linear iterations for the ith time step for the corresponding parameter $\boldsymbol{\mu}_j$.
2. Assemble all the snapshot matrices $\mathbf{B}_i(\boldsymbol{\mu}_j)$ into a singular snapshot matrix $\mathbf{S}_{\mathbf{B}}^j(\boldsymbol{\mu}_j)$ for each parameter $\boldsymbol{\mu}_j$. Specifically, stack the snapshot matrices $\mathbf{B}_i(\boldsymbol{\mu}_j)$ for all timesteps and the jth parameter $\boldsymbol{\mu}_j$ into a singular matrix representing all non-converged residuals:

$$\mathbf{S}_{\mathbf{B}}^j(\boldsymbol{\mu}_j) = \left[\mathbf{B}_1(\boldsymbol{\mu}_j), \mathbf{B}_2(\boldsymbol{\mu}_j), \ldots, \mathbf{B}_T(\boldsymbol{\mu}_j)\right]. \tag{56}$$

3. Compile all the snapshot matrices $\mathbf{S}_{\mathbf{B}}^j(\boldsymbol{\mu}_j)$ into a final non-converged residuals matrix for all training parameters:

$$\mathbf{S}_{\mathbf{R}} = \left[\mathbf{S}_{\mathbf{B}}^1(\boldsymbol{\mu}_1), \mathbf{S}_{\mathbf{B}}^2(\boldsymbol{\mu}_2), \ldots, \mathbf{S}_{\mathbf{B}}^P(\boldsymbol{\mu}_P)\right]. \tag{57}$$

4. Obtain the orthogonal basis $\boldsymbol{\Psi}_{\mathbf{R}}$:
   Perform a truncated SVD to obtain the orthogonal basis for their column spaces:

$$[\boldsymbol{\Psi}_{\mathbf{R}}, \bullet, \bullet] = \text{SVD}(\mathbf{S}_{\mathbf{R}}, \epsilon_{\mathbf{R}}). \tag{58}$$

   It is clear that

$$col(\boldsymbol{\Psi}_{\mathbf{R}}) \subseteq col(\boldsymbol{\Psi}_{\mathbf{J}}) \tag{59}$$

   (see Fig. 2 for schematic overview)

It is worth mentioning that, for highly nonlinear dynamical systems, the number of modes in the right ROB is expected to be higher than the number of non-linear iterations. This implies that the size of the $\mathbf{S}_{\mathbf{R}}$ matrix, which is constructed by gathering non-converged residuals for each parameter, will only increase in proportion to the number of non-linear iterations and is expected to be smaller than the size of the $\mathbf{S}_{\mathbf{J}}$ matrix, which is constructed using the Jacobian matrix projected onto the left ROB.

5.2. Comparing matrix sizes and computational costs in iterative procedures: The case of rectangular Petrov–Galerkin approach

Reduced-order modeling involves iterative procedures, for example, in the Newton–Raphson method, the matrix size plays a crucial role in determining both the computational cost and the solution approach.

Let us consider the matrix sizes involved in three different iterative schemes: the FOM, Galerkin ROM, and Petrov–Galerkin ROM. The FOM system involves a matrix

$$\mathbf{J}_t \in \mathbb{R}^{N\times N},$$

where N is the original dimensionality of the system. The Galerkin projection, on the other hand, employs the matrix

$$\boldsymbol{\Phi}^T \mathbf{J}_t \boldsymbol{\Phi} \in \mathbb{R}^{n\times n},$$

which results in a smaller system ($n \ll N$) that needs to be solved, leading to significant computational savings during the iterative procedure. It is worth noting that the full order model, Galerkin ROM, and LSPG ROM systems considered thus far have all been square systems. However, the Petrov–Galerkin projection requires solving a rectangular system of size

$$\boldsymbol{\Psi}^T \mathbf{J}_t \boldsymbol{\Phi} \in \mathbb{R}^{m \times n},$$

where $m \geq n$ (with $\mathcal{O}(m) = \mathcal{O}(n)$), leading to a minimization problem that needs to be addressed. One possible solution to this problem is QR factorization. Fig. 3 illustrates a schematic representation of the various systems and their resulting sizes. Additionally, Algorithm 5 presents the invariant left ROB PG-PROM algorithm, which differs from the general PG-PROM algorithm in Algorithm 2 as the left ROB $\boldsymbol{\Psi}$ is an input rather than an iteration-dependent variable. This feature brings significant computational savings in the subsequent hyper-reduction stage, as we will see later on. Despite the increased system size and associated computational cost, the Petrov–Galerkin projection can provide better accuracy than the Galerkin projection in certain situations.

Let us examine the computational costs tied to the training and online phases of the Galerkin, LSPG, and Petrov–Galerkin strategies more closely. Both the Galerkin and LSPG strategies involve a FOM assembly and solution for parameters P and their respective times T during the training phase. The complexity of the FOM assembly is $\mathcal{O}(L)$, and for the FOM solution, it is $\mathcal{O}(N^k)$, with $k \geq 1$ depending on the specific iterative solver employed for the sparse system. Then, a Singular Value Decomposition (SVD) compression is performed with a complexity of $\mathcal{O}(N(TP)n)$ using a randomized SVD. In the subsequent online phase, the FOM assembly is repeated, followed by a system projection operation, with complexities of $\mathcal{O}(N_0 \times n)$ and $\mathcal{O}(n^2 \times N)$ for sparse-dense and dense-dense matrix multiplication respectively, where N_0 represents the non-zero entries. The online phase concludes with a direct solution of a dense ROM, which is associated with a complexity of $\mathcal{O}(n^3)$. Conversely, the Petrov–Galerkin strategy introduces an additional training phase, which involves a repeated FOM assembly, system projection, and solution of the LSPG ROM for the same parameters P and their corresponding times T. This phase also requires an additional SVD compression operation, with a complexity of $\mathcal{O}(N(TPK)n)$, reflecting the sum of nonlinear iterations (K) in a residual-based approach, and once again employing a randomized SVD. Despite these heightened computational costs, this phase bolsters the HROM scheme by negating the need for a complementary mesh during the creation of the HROM model. The online phase mirrors that of the Galerkin and LSPG strategies but adjusts the ROM solution complexity to $\mathcal{O}(mn^2)$ (with $\mathcal{O}(m) = \mathcal{O}(n)$).

Algorithm 5: Newton-Raphson strategy for invariant left ROB PG-PROM.

Input: State variable $\mathbf{u}_{\text{ref}}$, right ROB $\boldsymbol{\Phi}$, and left ROB $\boldsymbol{\Psi}$
Output: State variable $\tilde{\mathbf{u}}_t$ and increment update $\boldsymbol{\Delta}\hat{\mathbf{u}}_t$
Initialize $\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(0)} = \mathbf{0}$, i.e., $\tilde{\mathbf{u}}_t^{(0)} = \mathbf{u}_{\text{ref}}$
for $k = 0, \ldots, K$ (convergence) **do**
  Evaluate residual $\mathbf{R}_t^{(k)} = \mathbf{R}_t|_{\tilde{\mathbf{u}}_t^{(k)}}$
  Evaluate Jacobian $\mathbf{J}_t^{(k)} = \frac{\partial \mathbf{R}_t}{\partial \tilde{\mathbf{u}}}\big|_{\tilde{\mathbf{u}}_t^{(k)}}$
  Compute $\mathbf{W}^{(k)} = \boldsymbol{\Psi}^T \mathbf{J}_t^{(k)} \boldsymbol{\Phi}$
  if $m \neq n$ **then**
    Compute QR decomposition $\mathbf{W}^{(k)} = \mathbf{Q}^{(k)} \mathbf{D}^{(k)}$
    Solve $\mathbf{D}^{(k)} \hat{\mathbf{p}}_t^{(k)} = -\mathbf{Q}^{T(k)} \boldsymbol{\Psi}^T \mathbf{R}_t^{(k)}$
  else
    Solve $\mathbf{W}^{(k)} \hat{\mathbf{p}}_t^{(k)} = -\boldsymbol{\Psi}^T \mathbf{R}_t^{(k)}$
  end if
  Update $\boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)} = \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k)} + \alpha_t^{(k)} \hat{\mathbf{p}}_t^{(k)}$
  Update $\tilde{\mathbf{u}}_t^{(k+1)} = \tilde{\mathbf{u}}_t^{(k)} + \boldsymbol{\Phi} \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{(k+1)}$
end for
$\boldsymbol{\Delta}\hat{\mathbf{u}}_t = \boldsymbol{\Phi} \sum_{k=1}^{K} \boldsymbol{\Delta}\hat{\mathbf{u}}_t^{k+1}$
$\tilde{\mathbf{u}}_t = \tilde{\mathbf{u}}_t^{(K+1)} \equiv \mathbf{u}_{\text{ref}} + \boldsymbol{\Delta}\hat{\mathbf{u}}_t$

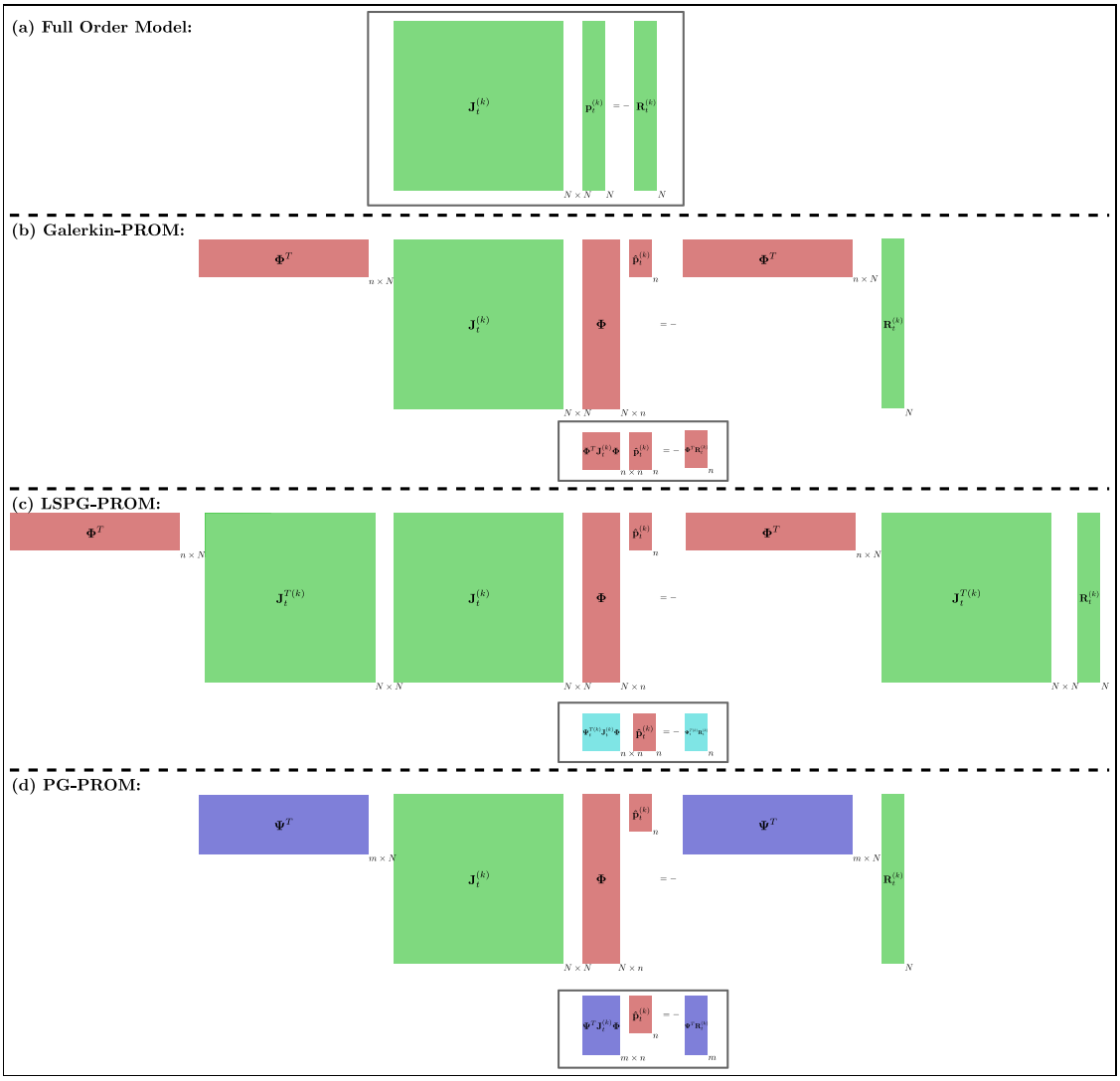


Fig. 3. This diagram illustrates the various dimensions of the systems that need to be addressed in different scenarios, as well as their corresponding projections. The four cases depicted are: (a) a full-order model, (b) a Galerkin-PROM, (c) a LPSG-PROM, and (d) a PG-PROM.

5.3. Hyper-reduction

The Newton–Raphson iterations for the invariant left ROB Petrov–Galerkin Projection-based Reduced Order Model with residual-based formulation, aforementioned: for $k = 1, \ldots, K$, we solve the following equations:

$$\boxed{\begin{aligned} \boldsymbol{\Psi}_{\mathbf{R}}^{T}\mathbf{J}_{t}^{(k)}(\tilde{\mathbf{u}}_{t}^{(k)};\boldsymbol{\mu})\boldsymbol{\Phi}\hat{\mathbf{p}}_{t}^{(k)} &= -\boldsymbol{\Psi}_{\mathbf{R}}^{T}\mathbf{R}_{t}^{(k)}(\tilde{\mathbf{u}}_{t}^{(k)};\boldsymbol{\mu}) \\ \boldsymbol{\Delta}\hat{\mathbf{u}}_{t}^{(k+1)} &= \boldsymbol{\Delta}\hat{\mathbf{u}}_{t}^{(k)} + \alpha_{t}^{(k)}\hat{\mathbf{p}}_{t}^{(k)} \\ \tilde{\mathbf{u}}_{t}^{(k+1)} &= \tilde{\mathbf{u}}_{t}^{(k)} + \boldsymbol{\Phi}\boldsymbol{\Delta}\hat{\mathbf{u}}_{t}^{(k+1)}. \end{aligned}} \tag{60}$$

We can highlight the difference between the variant and invariant left ROB approaches by revisiting Eq. (21), which, when substituted with the residual-based left ROB, gives:

$$\begin{aligned}\boldsymbol{\Psi}_{\mathbf{R}}^{T}\mathbf{R}_t &= \boldsymbol{\Psi}_{\mathbf{R}}^{T}\sum_{e=1}^{L}\mathbf{L}^{e^T}\mathbf{R}_t^e \\ &= \sum_{e=1}^{L}(\mathbf{L}^e\boldsymbol{\Psi}_{\mathbf{R}})^T\mathbf{R}_t^e \\ &= \boxed{\sum_{e=1}^{L}\boldsymbol{\Psi}_{\mathbf{R}}^{e^T}\mathbf{R}_t^e}.\end{aligned} \tag{61}$$

Comparing this to Eq. (33), the main difference in the LSPG assembling in the variant approach is the term $\bar{\mathbf{R}}_t^e$ (Eq. (36)). This term requires determining residual vectors for all elements that share nodes with element e (i.e., a patch of elements), which leads to the need to store a complementary mesh for the online evaluation of the LSPG-HPROM. In contrast, the left ROB in the PG-HPROM method is invariant and does not depend on surrounding elements. Therefore, the hyper-reduction scheme can be efficiently coupled with the ECM algorithm using an element-by-element construction and optimal mesh sampling.

6. Results

- Analysis of the nonlinear behavior of a cantilever beam subjected to prescribed forces in structural mechanics, with hyperelastic material properties described by the Kirchhoff Saint-Venant model.
- A transient rotating pulse convection–diffusion problem with convection dominance.
- Flow past a cylinder benchmark in fluid dynamics.

To evaluate the accuracy of the constructed PROMs and HPROMs, we measure the relative error between the solution state variables and their corresponding values obtained from the FOM and PROM-based models. At each ith time-step solution snapshot of the jth parameter configuration, the relative error is calculated as:

$$E_i(\mu_j) = \frac{\left\|\tilde{\mathbf{u}}_i(\mu_j) - \mathbf{u}_i(\mu_j)\right\|^2}{\left\|\mathbf{u}_i(\mu_j)\right\|^2}, \tag{62}$$

where $\tilde{\mathbf{u}}_i(\mu_j) \in \mathbb{R}^N$ and $\mathbf{u}_i(\mu_j) \in \mathbb{R}^N$ are the approximated and reference solutions of the state variable, respectively, at the ith time step for the jth parameter configuration. We also calculate the relative error for the entire set of solution snapshots, which is defined as:

$$E = \frac{\left\|\tilde{\boldsymbol{S}}_{\mathbf{u}} - \boldsymbol{S}_{\mathbf{u}}\right\|_F}{\left\|\boldsymbol{S}_{\mathbf{u}}\right\|_F}, \tag{63}$$

where

$$\tilde{\boldsymbol{S}}_{\mathbf{u}} = \left[\tilde{\mathbf{u}}_1(\boldsymbol{\mu}_1), \quad \tilde{\mathbf{u}}_2(\boldsymbol{\mu}_1), \quad \ldots, \quad \tilde{\mathbf{u}}_T(\boldsymbol{\mu}_1), \quad \tilde{\mathbf{u}}_1(\boldsymbol{\mu}_2), \quad \ldots, \quad \tilde{\mathbf{u}}_T(\boldsymbol{\mu}_2), \quad \ldots, \quad \tilde{\mathbf{u}}_T(\boldsymbol{\mu}_P)\right] \tag{64}$$

and

$$\boldsymbol{S}_{\mathbf{u}} = \left[\mathbf{u}_1(\boldsymbol{\mu}_1), \quad \mathbf{u}_2(\boldsymbol{\mu}_1), \quad \ldots, \quad \mathbf{u}_T(\boldsymbol{\mu}_1), \quad \mathbf{u}_1(\boldsymbol{\mu}_2), \quad \mathbf{u}_2(\boldsymbol{\mu}_2), \quad \ldots, \quad \mathbf{u}_T(\boldsymbol{\mu}_2), \quad \ldots, \quad \mathbf{u}_T(\boldsymbol{\mu}_P)\right] \tag{65}$$

are the snapshot matrices capturing the solution and approximation state variables across all time steps and parameter configurations.

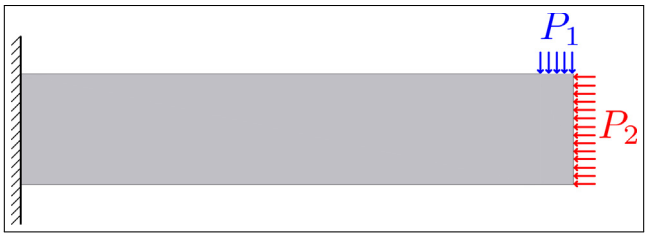


Fig. 4. Cantilever beam with two pressure loads.

6.1. Structural mechanics case

In this research, we analyze the nonlinear behavior of a cantilever beam in the field of structural mechanics, with the primary objective to compare the performance of Galerkin-PROMs, LSPG-PROMs, and PG-PROMs for systems governed by SPD operators. Additionally, the Jacobian-based and Residual-based approaches for selecting the left reduced order basis (ROB) $\mathbf{\Psi}$ are compared. The cantilever beam, subjected to prescribed forces, is assumed to possess hyperelastic material properties, as described by the Kirchhoff Saint-Venant model. The material properties include a Young's modulus of 206.9 GPa and a Poisson's ratio of 0.29. This beam is designed to undergo large displacements, with the problem being formulated using a Total Lagrangian formulation under a plane stress analysis setup. Two pressure loads, P_1 and P_2, are applied to the system. They are defined as $P_1 = c_1 * \sqrt{\alpha}$ and $P_2 = c_2 * \sqrt{\alpha}$ respectively, with $c_1 = 10^8$ Newtons and $c_2 = 10^7$ Newtons. These loads, directed in negative vertical and horizontal directions, are presented in Fig. 4. The loads are applied simultaneously with varying intensities over a series of steps to simulate the nonlinear behavior of the cantilever beam. The model will be trained for α values in the range of $[0.1, 2.0]$ and tested for extrapolated values of α in the range $(2.0, 4.0]$. As also depicted in Fig. 4, there is a fixed displacement imposed on the left boundary. The model for this analysis comprises 6954 finite elements with 3626 nodes, leading to 7252 degrees of freedom. This comprehensive model complexity aims to provide a precise simulation of the nonlinear behavior of the system under the conditions specified.

Reduction. For this analysis, specific tolerances were set to guide the reduction process. The tolerance used to determine the dimensionality of the latent variables $\hat{\mathbf{u}}$, which subsequently defines the number of columns in the right reduced order basis (ROB) $\mathbf{\Phi}$, was set at $\epsilon_u = 10^{-6}$. For the Petrov–Galerkin approaches, both Residual-based and Jacobian-based, the singular value decomposition (SVD) tolerance was established as $\epsilon_{\mathbf{R}} = \epsilon_{\mathbf{J}} = 10^{-6}$. This tolerance value serves to determine the size of the left ROB $\mathbf{\Psi}$. Lastly, the tolerance for the empirical cubature method (ECM), used across all projection strategies, was set to machine precision.

Discussion. The numerical investigation led to a latent approximation space $\mathbf{\Phi}$ of dimension 3 (number of modes) and a constraint space $\mathbf{\Psi}$ of dimension 7 (number of modes). The latter size was consistent in both Petrov–Galerkin methods, Jacobian-based and Residual-based, highlighting the uniformity in the constraint space selection.

Analyzing the overall L2 norms in both the training and testing phases, our models have delivered highly promising results. In the training phase (Table 1), the L2 norms between FOM, ROM, and HROM models were found to be on the order of 10^{-6} to 10^{-8} for both x and y displacements. This corresponds closely to our original error truncation tolerance set at 10^{-6}, thereby demonstrating excellent model fidelity in reproducing the training scenarios. Moreover, both the LSPG and Petrov–Galerkin strategies yielded similarly low L2 norms, reinforcing the residual-minimum property these methods aim to achieve. This result not only validates our proposed methodologies, but also highlights the underlying relationship between these projection strategies. In the testing phase (Table 2), where the models are applied to an extrapolated parametric space, the L2 norms remained low and within the same order of magnitude as the original error truncation tolerance. This result validates the robustness of our models, as they maintain a high degree of accuracy even when extrapolated beyond their training data (see Figs. 5 and 6).

The HROMs selection process resulted in the following distribution of elements: Galerkin strategy yielded 20 elements (0.29% of the total 6954 elements), LSPG strategy yielded 35 elements (0.50% of the total), Petrov–Galerkin Residual-based strategy selected 68 elements (0.98% of the total), and Petrov–Galerkin Jacobian-based strategy selected 54 elements (0.78% of the total). It is worth noting that although the LSPG strategy initially

Table 1
Overall L2 norms for training phase.

Strategy	Overall L2 Norm (Training)			Variable
	FOM vs ROM	ROM vs HROM	FOM vs HROM	
Galerkin	2.125×10^{-8}	9.630×10^{-13}	2.125×10^{-8}	Displacement x
	4.089×10^{-9}	1.060×10^{-12}	4.089×10^{-9}	Displacement y
LSPG	1.257×10^{-6}	1.251×10^{-9}	1.257×10^{-6}	Displacement x
	1.136×10^{-6}	1.173×10^{-9}	1.136×10^{-6}	Displacement y
Petrov–Galerkin Res.	9.731×10^{-7}	1.434×10^{-8}	9.587×10^{-7}	Displacement x
	8.826×10^{-7}	1.325×10^{-8}	8.693×10^{-7}	Displacement y
Petrov–Galerkin Jac.	9.060×10^{-7}	1.159×10^{-8}	8.946×10^{-7}	Displacement x
	7.814×10^{-7}	1.021×10^{-8}	7.712×10^{-7}	Displacement y

Table 2
Overall L2 norms for testing phase.

Strategy	Overall L2 Norm (Testing)			Variable
	FOM vs ROM	ROM vs HROM	FOM vs HROM	
Galerkin	3.331×10^{-7}	2.748×10^{-11}	3.331×10^{-7}	Displacement x
	6.843×10^{-8}	3.326×10^{-11}	6.842×10^{-8}	Displacement y
LSPG	2.095×10^{-5}	1.230×10^{-8}	2.094×10^{-5}	Displacement x
	1.968×10^{-5}	1.178×10^{-8}	1.967×10^{-5}	Displacement y
Petrov–Galerkin Res.	1.618×10^{-5}	2.339×10^{-7}	1.595×10^{-5}	Displacement x
	1.528×10^{-5}	2.243×10^{-7}	1.506×10^{-5}	Displacement y
Petrov–Galerkin Jac.	1.508×10^{-5}	2.279×10^{-7}	1.486×10^{-5}	Displacement x
	1.354×10^{-5}	2.020×10^{-7}	1.334×10^{-5}	Displacement y

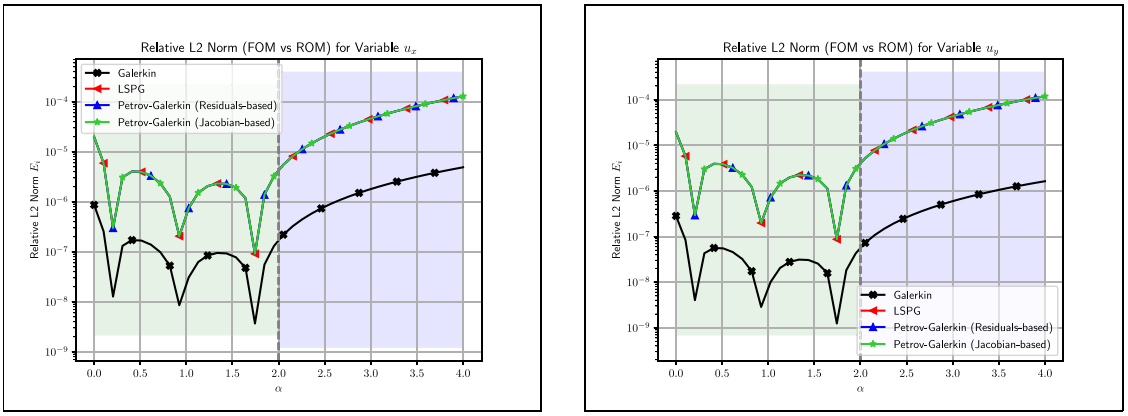
Table 3
Speedups for ROM and HROM using different strategies.

Strategy	Total speedup
ROM Galerkin	9.92
HROM Galerkin	612.13
ROM LSPG	13.32
HROM LSPG	119.85
ROM Petrov-Galerkin Res.	7.70
HROM Petrov-Galerkin Res.	164.70
ROM Petrov-Galerkin Jac.	8.56
HROM Petrov-Galerkin Jac.	228.86

appears to require fewer selected elements than the Petrov–Galerkin strategies, it demands information from the surrounding elements and thus necessitates a complementary mesh. This increases the effective number of elements from 35 to 293, as shown in Fig. 7. Consequently, despite the initially smaller selection, LSPG yields a considerably lower speedup when compared to the other strategies (see Table 3). These results elucidate the significant speedups achieved when deploying HROM strategies compared to their traditional ROM counterparts.

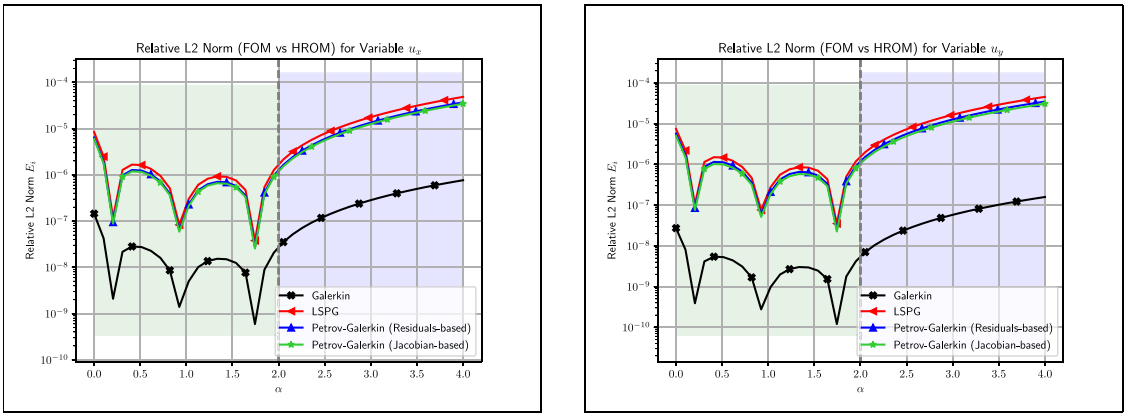
Our numerical results highlight that, particularly for problems exhibiting symmetric positive-definite (SPD) operators like the one studied in this case, the Galerkin approach consistently provides superior results and speed-ups. However, the reader must not overlook the primary goal of this work, which is to propose an equivalent projection strategy to LSPG, one that utilizes a fixed left reduced order basis, $\boldsymbol{\Psi}$, and obviates the necessity of the complementary mesh.

Noteworthy is the contrast in the speed-ups achieved: we observe a transition from an LSPG HROM speed-up of 119.85 to a Petrov–Galerkin speed-up of 164.70 for the Residuals-based approach and 228.86 for the Jacobian-based method. This significant impact in terms of computational efficiency is pronounced even for this relatively simple and coarse problem. Another important aspect to discuss is the relative effort required to train the HROM models from



(a) Displacement in x (b) Displacement in y

Fig. 5. Relative L2 Norm between the FOM and ROM against the scaling parameter α. The vertical line at $\alpha = 2.0$ denotes the boundary between the training ($\alpha \in [0.0, 2.0]$, shaded green) and testing phases ($\alpha \in (2.0, 4.0]$, shaded blue). Each line represents a different strategy used in the model reduction process. The subfigures represent two distinct variables of interest: (a) Displacement in x, and (b) Displacement in y. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)



(a) Displacement in x (b) Displacement in y

Fig. 6. Relative L2 Norm between the FOM and the HROM against the scaling parameter α. The vertical line at $\alpha = 2.0$ denotes the boundary between the training ($\alpha \in [0.0, 2.0]$, shaded green) and testing phases ($\alpha \in (2.0, 4.0]$, shaded blue). Each line represents a different strategy used in the model reduction process. The subfigures represent two distinct variables of interest: (a) Displacement in x, and (b) Displacement in y. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the FOM. In this specific case, if we consider the effort for training the Galerkin strategy as a reference, we observe that the LSPG strategy has a slightly higher ratio of 1.01 to the Galerkin, showing a similar efficiency in training the model. On the other hand, the Petrov–Galerkin strategies present a higher ratio of 1.38. This increase in training effort for the Petrov–Galerkin strategies originates from the second training phase, which is required to determine the optimal fixed left ROB $\boldsymbol{\Psi}$. This underscores the trade-off in computational efficiency when selecting a modeling strategy. In this case, the reduction in elements needed for the Petrov–Galerkin methods (which eliminates the need for a complementary mesh, unlike LSPG) comes at the cost of increased computational effort during training.

Moving forward, the aim is to investigate how these findings extrapolate to problems with finer meshes and different physics. We believe the strategies presented in this work will be invaluable tools in addressing more complex, real-world problems in structural mechanics and beyond.

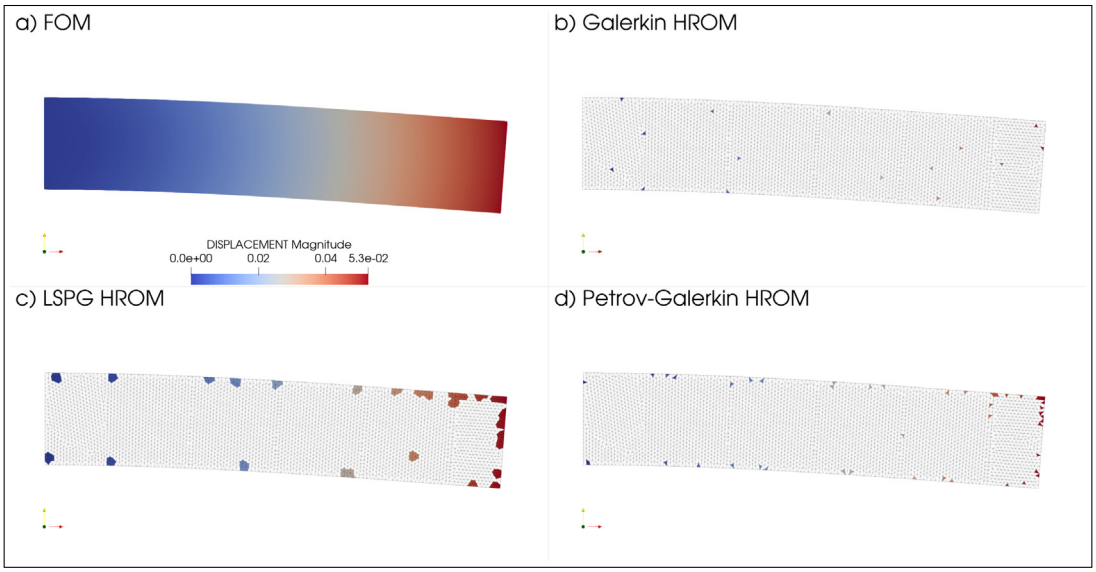


Fig. 7. Comparison of HROM solutions for different strategies in the context of the cantilever beam: (a) FOM, (b) Galerkin HROM, (c) LSPG HROM, (d) Petrov–Galerkin HROM. The figure also exhibits the different HROM meshes used for each strategy.

6.2. Convection–diffusion case

In this study, we solved a 2D convection–diffusion problem on a non-uniform mesh comprising 23204 elements and 11603 nodes (degrees of freedom). This problem, characterized by a transient rotating pulse, is adapted from [28]. The transient convection–diffusion equation is expressed as:

$$\frac{\partial u}{\partial t} + \mathbf{a} \cdot \nabla u - \nabla \cdot (\varepsilon \nabla u) = s \quad \text{in } \Omega = [0, 1] \times [0, 1]. \tag{66}$$

In this equation, u represents the unknown scalar field, $\frac{\partial u}{\partial t}$ is the rate of change of this field over time, and $\mathbf{a} \cdot \nabla u$ is the convective term where $\mathbf{a} = (-y, x)$ is the velocity field. The term $\nabla \cdot (\varepsilon \nabla u)$ represents diffusion, and s is the source term defined as:

$$s = \begin{cases} 10 \times \exp\left(-50\left(x^2 + y^2 - \frac{1}{2}\right)^2\right) & \text{if } \sqrt{x^2 + y^2} < 1, \\ 0 & \text{otherwise.} \end{cases} \tag{67}$$

The boundary conditions are set as $u = 0$ on $\partial\Omega$ (the boundary of Ω), and $u = 0$ at $t = 0$.

This problem is chosen due to its convection-dominant nature. For further details on the problem setup, please refer to [28].

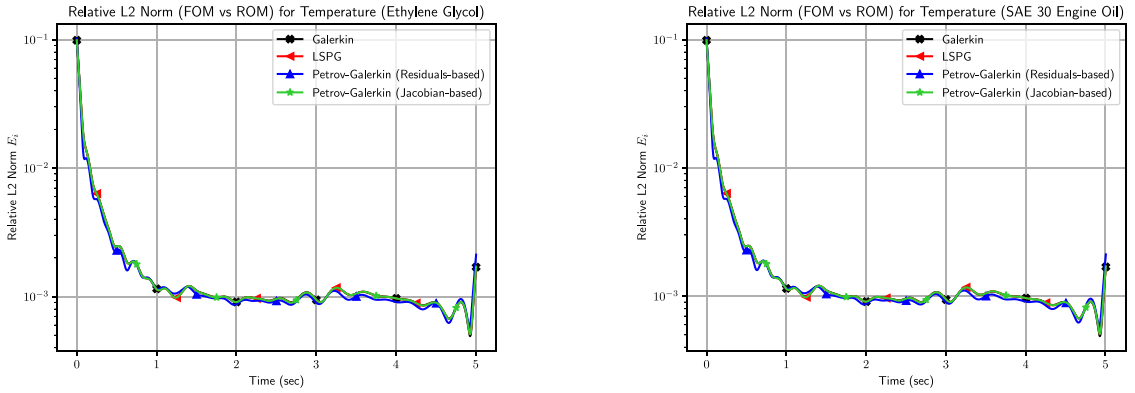
The reduced order model was trained using properties of two different materials: Ethylene Glycol and SAE 30 Engine Oil. Ethylene Glycol has a density of approximately 1110 kg/m^3, a thermal conductivity of 0.253 W/(m*K), and a specific heat of 2412 J/(kg*K). SAE 30 Engine Oil, on the other hand, has a density of approximately 875 kg/m^3, a thermal conductivity of 0.15 W/(m*K), and a specific heat of 2092 J/(kg*K).

In the testing phase, the reduced order model was extrapolated to a third material, Glycerol. Glycerol has a density of approximately 1260 kg/m^3, a thermal conductivity of 0.286 W/(m*K), and a specific heat of 2430 J/(kg*K). The primary goal of this experiment is to assess the ability of the reduced order model to generalize over varying material parameters. The time for the convection–diffusion problem was defined from 0 to 5 s.

Reduction. The reduction process for the convection–diffusion problem was carried out with a tolerance level of $\epsilon_u = 10^{-3}$, which was used to determine the dimensionality of the latent variables $\hat{\mathbf{u}}$. Under this tolerance, the number of modes in the right reduced order basis (ROB) $\mathbf{\Phi}$ was 18. The Petrov–Galerkin methods, with singular value decomposition (SVD) tolerances of $\epsilon_{\mathbf{R}} = \epsilon_{\mathbf{J}} = 10^{-3}$, resulted in a left ROB, $\mathbf{\Psi}$, comprising 18 modes for the

Table 4
Mean relative L2 norms for the training phase.

Strategy	FOM vs ROM	ROM vs HROM	FOM vs HROM
Galerkin	2.323×10^{-3}	1.35×10^{-6}	1.089×10^{-3}
LSPG	2.326×10^{-3}	1.399×10^{-5}	1.640×10^{-3}
Petrov–Galerkin Res.	2.180×10^{-3}	1.635×10^{-4}	1.267×10^{-3}
Petrov–Galerkin Jac.	2.326×10^{-3}	1.335×10^{-6}	1.230×10^{-3}



(a) Ethylene Glycol (b) SAE 30 Engine Oil

Fig. 8. Relative L2 Norm between the FOM and ROM for different training materials. Each line represents a different strategy used in the model reduction process. The subfigures represent two distinct materials: (a) Ethylene Glycol, and (b) SAE 30 Engine Oil.

Jacobian-based approach and 19 modes for the Residual-based approach. The empirical cubature method (ECM), which was utilized in all the projection strategies, was configured with a tolerance of 10^{-6}.

Discussion. Our model's initial validation utilized training materials, specifically Ethylene Glycol and SAE 30 Engine Oil. The resultant mean relative L2 errors across different model reduction strategies are encapsulated in the Table 4.

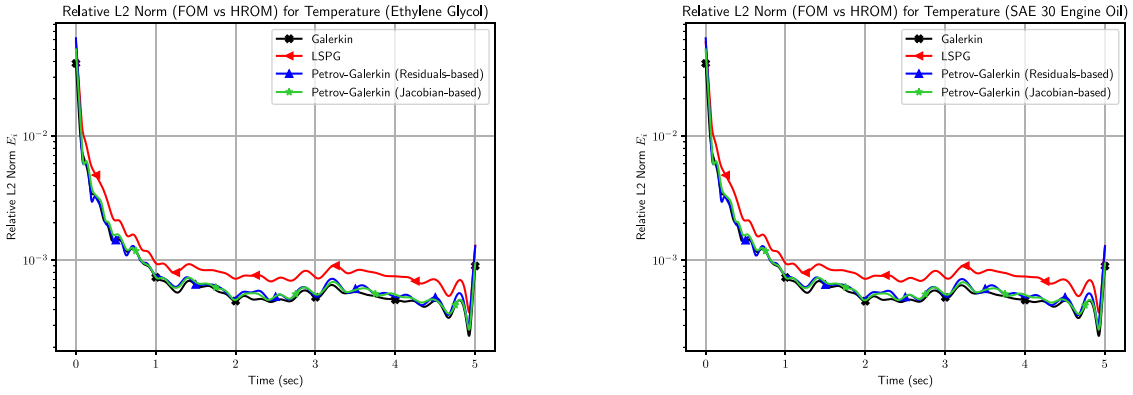
In contrast to the structural mechanics case, the Galerkin method, impacted by the non-SPD nature of the problem, fails to yield a clear minimum solution. Nonetheless, an interesting observation from the results is the convergence of Petrov–Galerkin strategies and the LSPG method to similar error magnitudes. Illustrating the evolution of the mean relative L2 error for varying reduction strategies concerning the temperature variable, Figs. 8 and 9 further highlight the implications of the problem not being SPD.

Remarkably, the Petrov–Galerkin strategies not only match the LSPG's error values but also outperform them within an HROM context further presented. The effectiveness and aptness of the Petrov–Galerkin strategies in handling such complicated problems are thereby accentuated.

Delving into the HROM selection process, it yielded the following element distribution: the Galerkin strategy incorporated 365 elements (representing 1.57% of the total 23204 elements), the LSPG strategy included 358 elements (1.54% of the total), the Petrov–Galerkin Residual-based strategy integrated 462 elements (1.99% of the total), and the Petrov–Galerkin Jacobian-based strategy adopted 439 elements (1.89% of the total). It is noteworthy that, despite seemingly requiring fewer elements, the LSPG strategy depends on information from surrounding elements, necessitating a complementary mesh. The LSPG strategy further results in a marked increase in the effective number of elements, growing from 358 to 3265, approximately 9.1 times the original selection. This inflated selection constitutes 14.07% of the original FOM with 23204 elements, a significant increase from the initial 1.54%, as depicted in Fig. 10.

In contrast, the Petrov–Galerkin strategies hold an advantage as they prevent this rise in elements while still providing a solution with a minimum residual comparable to the LSPG strategy.

Having delved into the complexities of element distribution within different strategies, let us now shift our focus to another critical metric: the total speedups achieved for the ROMs and HROMs across these strategies. This will



(a) Ethylene Glycol (b) SAE 30 Engine Oil

Fig. 9. Relative L2 Norm between the FOM and the HROM for different training materials. Each line represents a different strategy used in the model reduction process. The subfigures represent two distinct materials: (a) Ethylene Glycol, and (b) SAE 30 Engine Oil.

Table 5
Average speedups for ROM and HROM using different strategies, and the percentage of elements selected for HROM with respect to the FOM.

Strategy	Total speedup (average over materials)
ROM Galerkin	4.04
HROM Galerkin	49.0
ROM LSPG	4.33
HROM LSPG	21.35
ROM Petrov-Galerkin Res.	3.63
HROM Petrov-Galerkin Res.	35.39
ROM Petrov-Galerkin Jac.	3.64
HROM Petrov-Galerkin Jac.	37.44

Table 6
Mean relative L2 norms for the testing phase with glycerol.

Strategy	FOM vs ROM	ROM vs HROM	FOM vs HROM
Galerkin	2.324×10^{-3}	1.15×10^{-6}	1.090×10^{-3}
LSPG	2.327×10^{-3}	1.41×10^{-5}	1.640×10^{-3}
Petrov–Galerkin Res.	2.167×10^{-3}	1.22×10^{-14}	1.267×10^{-3}
Petrov–Galerkin Jac.	2.326×10^{-3}	6.07×10^{-15}	1.231×10^{-3}

provide us with another important perspective on the comparative performance of these methods. Table 5 below encapsulates the total average speedups for the ROM and HROM using different strategies.

After establishing our model's performance with the training materials, we expanded our analysis to encompass a material distinct from the training set, specifically, Glycerol. This rigorous test allows us to evaluate our model's ability in predicting the behavior of materials beyond its training repertoire. Delving into the comparative examination of mean relative L2 norms for different model reduction strategies applied to Glycerol, the Table 6 below encapsulates these findings. It lays out the mean relative L2 norms for the temperature variable. Our exploration of Glycerol test results underlines the robust performance of our reduced order model across diverse materials with comparable properties. Figs. 11(a) ans 11(b) visualize the trajectory of the mean relative L2 error for various reduction strategies in relation to the temperature variable of the test material, Glycerol.

While the Galerkin strategy for HROM marks high speedup in this non-SPD problem and provides a benchmark, we must heed the fact that the Galerkin method does not guarantee convergence to the minimum solution for all non-SPD problems. Consequently, it could potentially induce instabilities [19]. Despite its commendable performance in the given context, its reliability may falter in other scenarios. Our prime interest lies in comparing and contrasting the LSPG strategy with the Petrov–Galerkin strategies. At first glance, the LSPG strategy may appear more resourceful

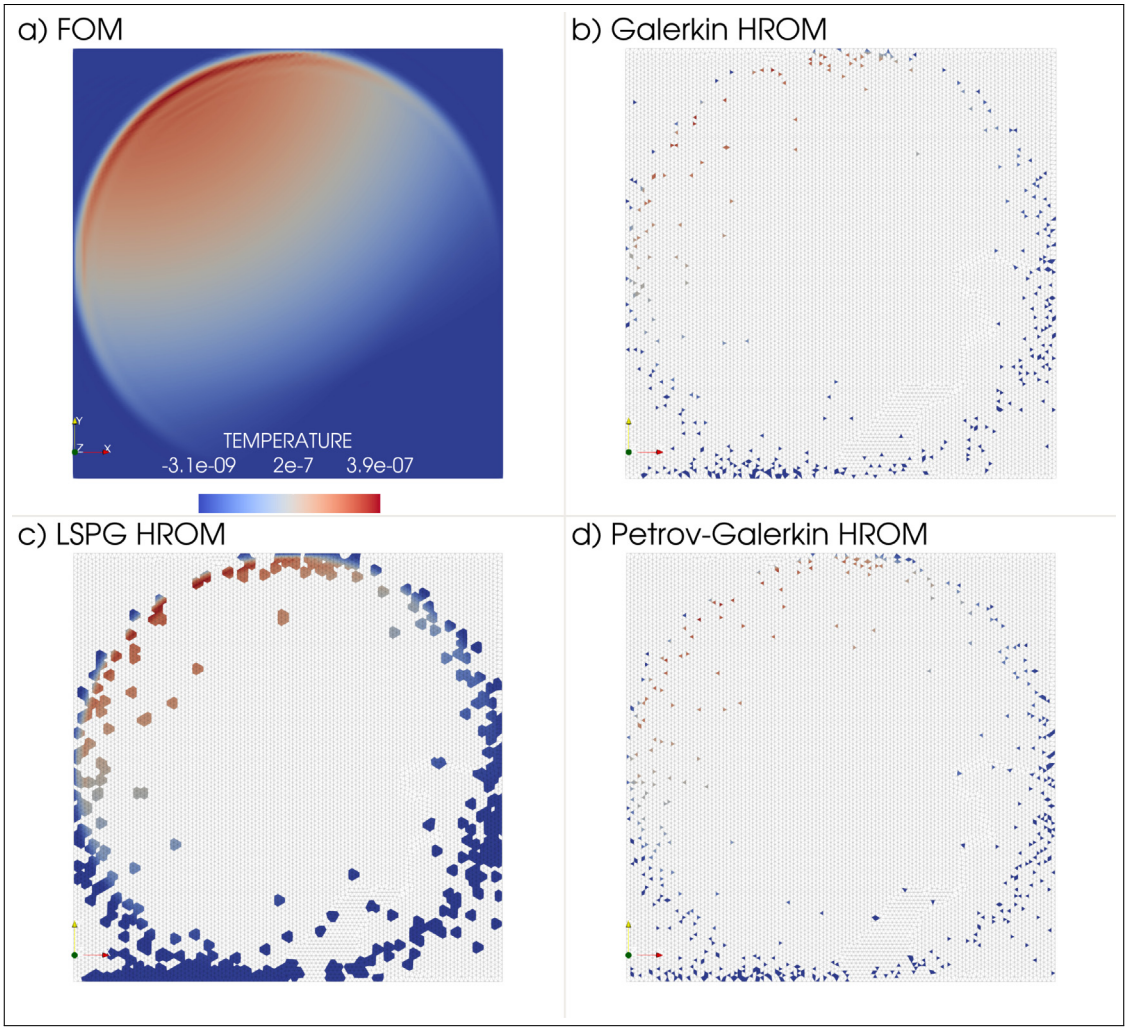


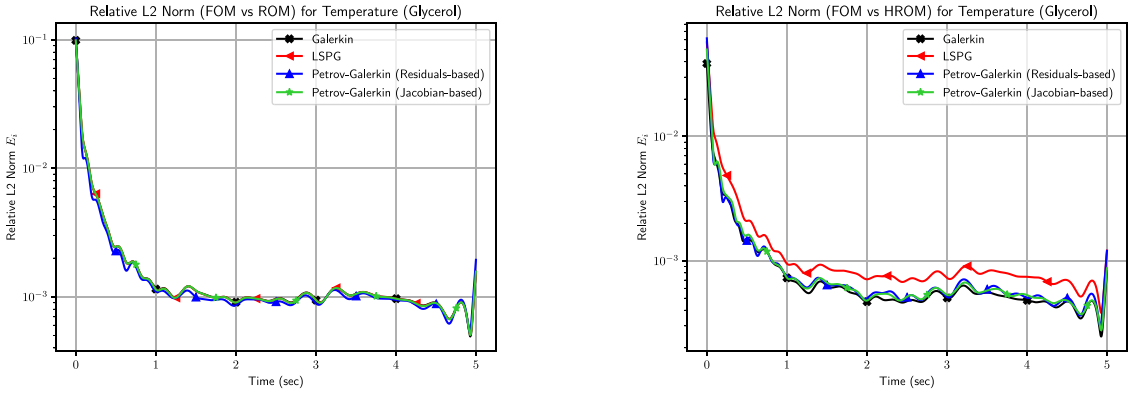
Fig. 10. Comparison of HROM solutions for different strategies in the context of the transient rotating pulse (Ethylene Glycol): (a) FOM, (b) Galerkin HROM, (c) LSPG HROM, (d) Petrov–Galerkin HROM. The figure also exhibits the different HROM meshes used for each strategy.

due to its more economical selection of elements from the FOM. However, its need for a larger effective selection driven by the necessary complementary mesh eventually cuts into its computational speedup.

Conversely, the Petrov–Galerkin strategies deliver substantial speedups while harnessing a relatively lean fraction of FOM elements. Impressively, these strategies achieve a solution fidelity comparable to the LSPG method but boast superior computational efficiency. This performance delineates a desirable balance between computational speedup and approximation accuracy, thereby hinting at the potential edge of Petrov–Galerkin strategies in non-SPD problem settings.

6.3. Fluid dynamics case

The 2D CFD simulation for the incompressible Navier–Stokes equations was performed to solve the well-known flow past a cylinder benchmark for specific base velocities. The problem geometry consists of a 5×2 m channel with a non-slip cylinder of 0.2 m diameter located at coordinates (1.25,0.5). The top and bottom walls are also non-slip, and the pressure is fixed along the right edge. The time-dependent parabolic inlet function is applied at



(a) FOM vs ROM

(b) FOM vs HROM

Fig. 11. Relative L2 Norm between the FOM and ROM or HROM for the testing material Glycerol. Each line represents a different strategy used in the model reduction process.

Table 7
The calculated Reynolds numbers for each base velocity. The test scenario ($v_b = 5.0\ \frac{\text{m}}{\text{s}}$) is shaded in gray.

v_b ($\frac{\text{m}}{\text{s}}$)	Re
4.0	66.7
5.0	83.3
6.0	100

the left edge, with a sinusoidal ramp-up applied to the inlet function from 0.0 to 1.0 s. This inlet function is defined as:

$$v(t) = \begin{cases} v_b y(1-y) sin(\frac{\pi t}{2}), & t \in [0, 1) \\ v_b y(1-y), & t \in [1, T] \end{cases}$$

For the monitoring of the results, a probe is positioned downstream from the cylinder's trailing edge at coordinates (1.43, 0.52). This probe serves to gather key data from the flow dynamics.

The simulations are run with base velocities (v_b) of $4.0\frac{\text{m}}{\text{s}}$ and $6.0\frac{\text{m}}{\text{s}}$ for a total of 30 s for the training phase, and with a base velocity of $5.0\frac{\text{m}}{\text{s}}$ for 40 s during the testing phase. These base velocities correspond to different Reynolds numbers, calculated using the formula:

$$Re = \frac{v_{avg} * \rho * D}{\eta},$$

where $\eta = 0.002\frac{\text{kg}}{\text{m s}}$ is the dynamic viscosity, $\rho = 1\frac{\text{kg}}{\text{m}^3}$ is the fluid density, $D = 0.2$ m is the diameter of the cylinder, and v_{avg} is the average velocity, calculated as $\frac{v_b}{6}$. The calculated Reynolds numbers for each base velocity are presented in Table 7.

The time step is 0.1 s, and the mesh consists of 68298 linear triangular elements with 34149 nodes (102447 degrees of freedom). Fig. 12 illustrates the problem setup and boundary conditions. By simulating this benchmark, we aim to evaluate the ability of our reduced-order models to reconstruct the flow behavior at different Reynolds numbers, derived from various base velocities.

Reduction. In this specific analysis, tolerance levels were carefully selected to obtain a sufficiently good approximation of the fluid-dynamics problem at hand, to avoid introducing high phase and amplitude errors. A tolerance level of $\epsilon_u = 10^{-4}$ was adopted to ascertain the dimensionality of the latent variables $\hat{\mathbf{u}}$. This resulted in the construction of a right ROB $\mathbf{\Phi}$ that incorporated 98 modes. For the Petrov–Galerkin approach, particularly the Residual-based method, a SVD tolerance of $\epsilon_{\mathbf{R}} = 5 \times 10^{-4}$ was employed to prevent the left ROB from becoming excessively

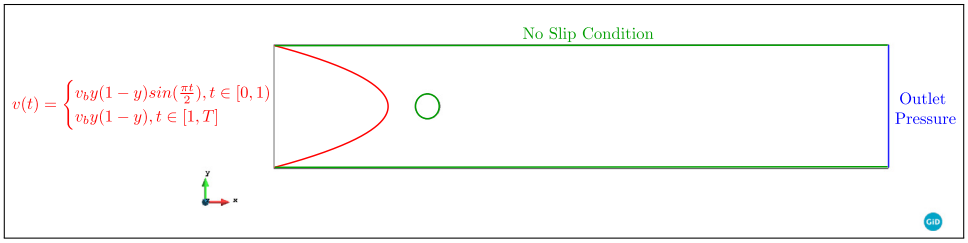


Fig. 12. Flow past a cylinder 2D.

large, and yielded a left ROB $\boldsymbol{\Psi}$ consisting of 153 modes. For the left ROB, only the Residual-based approach was employed in this case to minimize the memory load during the second training phase. This decision was strategic, as the Jacobian-based method would have resulted in a snapshots matrix for the left ROB growing linearly with the number of modes in the right ROB. In contrast, the Residual-based approach's snapshots matrix expands linearly with the number of nonlinear iterations, which are significantly fewer in this case than the number of modes. This strategy facilitated a more efficient training process without sacrificing the accuracy of the results. The empirical cubature method (ECM), deployed across all projection strategies, was configured with a tolerance of 10^{-5}.

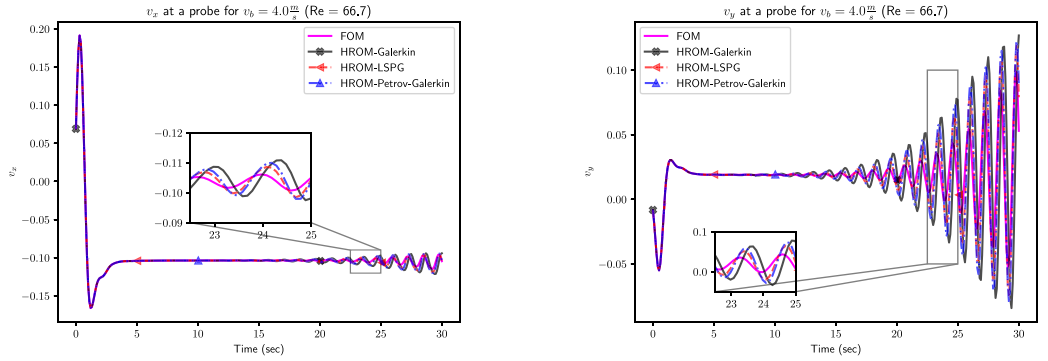
Discussion. With the objective of appraising the resilience and accuracy of our training model, we embarked on a validation study, employing the training parameters v_b of $4.0\frac{\text{m}}{\text{s}}$ ($Re = 66.7$) and $6.0\frac{\text{m}}{\text{s}}$ ($Re = 100$). These specific values were judiciously selected to encompass a varied, yet illustrative range of the prospective parameter space wherein the model would be anticipated to perform.

In this context, it is important to highlight certain aspects of the outcome that stem directly from the unique properties of the boundary conditions (e.g. the inlet ramp-up) and parametric settings for this case. Given the complexity of the problem, the model did not showcase a substantial reduction in the information required for an accurate representation. Specifically, out of 600 snapshots collected for the training set, the optimal right ROB required 98 modes to accurately represent both the phase and amplitude of the dynamics, devoid of high spikes and significant errors. This translates into the need for roughly one-sixth of the total information of the problem. While such a requirement may lead to potential limitations in the performance of both the ROM and the HROM, it is pivotal to note that these outcomes do not undermine the value of our research. Rather, they provide an opportunity to gain insights into the nature of complex systems.

Moreover, the primary intention of this paper is not focused on achieving the highest efficiency of information reduction, but rather on demonstrating and accentuating the performance of the proposed Petrov–Galerkin HROM in comparison to the LSPG approach for a general case. Even under less-than-optimal circumstances, our findings shed valuable light on the versatility and potential of the proposed HROM framework. One important aspect to highlight is the relative performance of the different reduced order models. Throughout the simulation, it was observed that both the LSPG and Petrov–Galerkin cases consistently presented results more closely aligned with those of the FOM, compared to the Galerkin ROM. This outcome underscores the superior performance of the Petrov–Galerkin and LSPG models in replicating the detailed fluid dynamics captured by the FOM, even under vortical flow conditions.

Furthermore, it is noteworthy that as the size of the left ROB is increased for the Petrov–Galerkin model, its outcomes should progressively align with those of the LSPG model. In other words, with a sufficiently large left ROB, the Petrov–Galerkin model would essentially replicate the LSPG results, underlining the inherent congruence between these two ROM methodologies when deployed with adequate basis sets. This characteristic reinforces their capability for delivering reliable and precise simulations, consistent with the FOM, for a wide range of fluid dynamics problems. The outcomes of these comparisons are visually represented in Figs. 13(a), 13(b), 14(a), and 14(b).

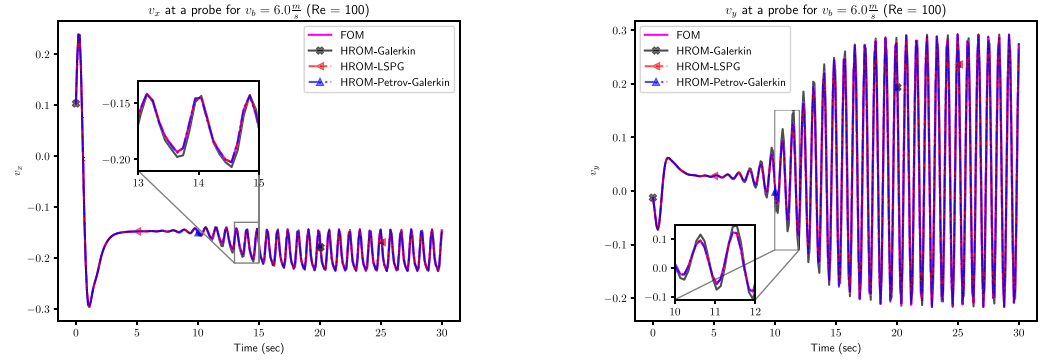
The HROM selection process yielded the following element distribution: the Galerkin strategy incorporated 6216 elements (representing about 9.10% of the total 68298 elements in the FOM), the LSPG strategy included 5914 elements (8.66% of the total), and the Petrov–Galerkin strategy integrated 5829 elements (about 8.53% of the total). Though the LSPG strategy seems to require fewer elements, it leverages information from surrounding elements, hence necessitating more elements in a complementary mesh. Consequently, the effective number of elements



(a) Velocity component in the x-direction at a selected probe node for the FOM and various HROM strategies ($v_b = 4.0\frac{m}{s}$)

(b) Velocity component in the y-direction at a selected probe node for the FOM and various HROM strategies ($v_b = 4.0\frac{m}{s}$)

Fig. 13. Velocity profiles at a selected probe node for the FOM and HROM using Galerkin, LSPG, and Petrov–Galerkin strategies. The velocities correspond to a Reynolds number of 66.7 ($v_b = 4.0\frac{\mathrm{m}}{\mathrm{s}}$) and represent a training parameter. Zoomed-in regions are provided to visualize specific details.



(a) Velocity component in the x-direction at a selected probe node for the FOM and various HROM strategies ($v_b = 6.0\frac{m}{s}$)

(b) Velocity component in the y-direction at a selected probe node for the FOM and various HROM strategies ($v_b = 6.0\frac{m}{s}$)

Fig. 14. Velocity profiles at a selected probe node for the FOM and HROM using Galerkin, LSPG, and Petrov–Galerkin strategies. The velocities correspond to a Reynolds number of 100 ($v_b = 6.0\frac{\mathrm{m}}{\mathrm{s}}$) and represent a training parameter. Zoomed-in regions are provided to visualize specific details.

expands from 5914 to 32838, about 5.55 times the original selection, constituting 48.07% of the original FOM. This surge represents a substantial increase from the initial 8.66%, as depicted in Fig. 15. In contrast, the Petrov–Galerkin strategies avert this surge in elements while still providing a solution with a minimum residual comparable to the LSPG strategy.

With a comprehensive understanding of the element distribution across different strategies, it is equally important to delve into another significant performance metric: the total speedups achieved for the HROM across these strategies. This analysis will provide us with a valuable perspective on the comparative efficiency of these methods. Table 8 encapsulates the total speedups for the HROM using different strategies.

The speedups depicted in Table 8 attest to the efficiency and effectiveness of these strategies within an HROM context. Notably, the Petrov–Galerkin strategy outperforms the LSPG strategy in terms of speedup, despite delivering comparable results. This improved efficiency can be attributed to Petrov–Galerkin's ability to bypass the need for a complementary mesh, offering significant advantages when selecting an optimal strategy for problems exhibiting non-SPD operators.

To further assess the performance of our model, we conducted additional tests for a different inflow velocity of $v_b = 5.0\frac{\mathrm{m}}{\mathrm{s}}$, corresponding to a Reynolds number of 83.3. This lies between the two training Reynolds numbers,

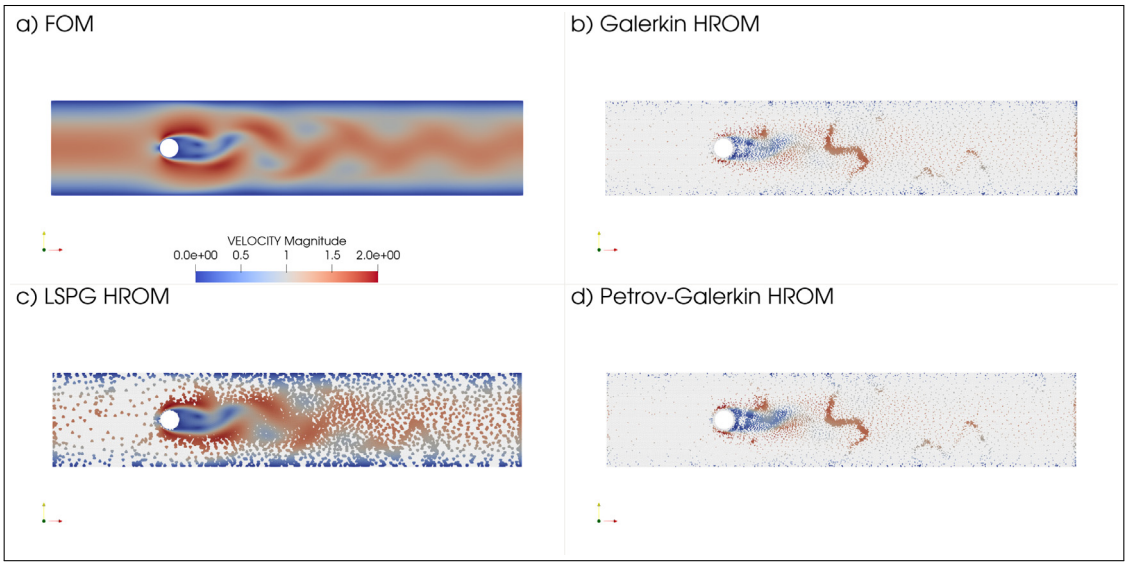


Fig. 15. Comparison of HROM solutions for different strategies in the context of the transient flow past a cylinder ($v_b = 6.0\frac{\text{m}}{\text{s}}$): (a) FOM, (b) Galerkin HROM, (c) LSPG HROM, (d) Petrov–Galerkin HROM. The figure also showcases the different HROM meshes used for each strategy.

Table 8
Total speedups for HROM using different strategies.

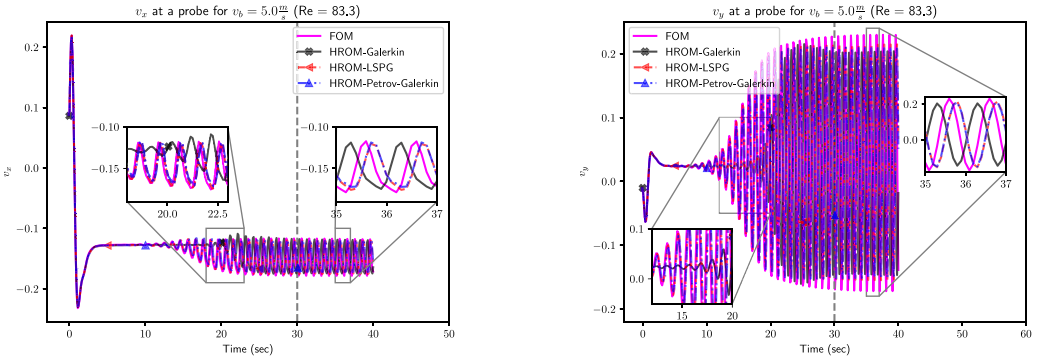
Strategy	Total speedup
HROM Galerkin	3.29
HROM LSPG	1.81
HROM Petrov-Galerkin	3.15

allowing us to examine the capability of our model to interpolate for a Reynolds number not explicitly included in the training set. Additionally, we tested the ability of our model to extrapolate in time for another 10 s, reaching up to 40 s instead of the 30 s it was trained on. This evaluation is critical to determining the robustness and adaptability of our model to predict future states beyond its training horizon. The results of these tests are visually represented in Figs. 16(a) and 16(b).

The figures reveal that the LSPG and Petrov–Galerkin strategies more accurately reproduce the phase and amplitude of the FOM. In contrast, the Galerkin strategy takes more time to reach the fully developed flow, shows higher amplitude errors, and experiences a noticeable phase shift. This underscores the effectiveness of the LSPG and Petrov–Galerkin strategies in accurately capturing the system dynamics, even under interpolation and extrapolation scenarios.

7. Conclusions

In this study, we have proposed an innovative approach that blends two key components: an invariant left ROB $\boldsymbol{\Psi}$ and its natural integration with the ECM hyper-reduction method. Crucially, our framework allows the left ROB to be different from the right ROB, providing enhanced flexibility in the hyper-reduction of nonlinear methods. Our model demonstrated significant improvements in accuracy, particularly for problems with non-SPD operators. Additionally, it exhibited remarkable efficiency in generating hyper-reduced models for PG-PROMs while preserving the minimum-residual optimality offered by LSPG projection. In the diverse range of problems studied, including structural mechanics (SPD), convection–diffusion (non-SPD), and fluid dynamics (non-SPD), our model showcased robust performance. A noteworthy aspect of our findings was the comparable performance of Petrov–Galerkin strategies to LSPG, despite the former exhibiting superior computational speedups. Petrov–Galerkin strategies also



(a) Velocity component in the x-direction at a selected probe node for the FOM and various HROM strategies ($v_b = 5.0\frac{m}{s}$)

(b) Velocity component in the y-direction at a selected probe node for the FOM and various HROM strategies ($v_b = 5.0\frac{m}{s}$)

Fig. 16. Velocity profiles at a selected probe node for the FOM and HROM using Galerkin, LSPG, and Petrov–Galerkin strategies. The velocities correspond to a Reynolds number of 83.3 ($v_b = 5.0\frac{\mathrm{m}}{\mathrm{s}}$) and represent a testing parameter. Zoomed-in regions are provided to visualize specific details. The vertical dashed gray line indicates the edge of the training horizon (time = 30 s).

managed to bypass the surge in the requirement of elements, a limitation observed in the LSPG approach due to the need for a complementary mesh. Although the introduction of our proposed PG-HPROM necessitates an additional offline training stage, its efficiency in terms of the number of selected elements equals that of Galerkin HPROMs, eliminating the need for a complementary mesh. Importantly, the model's robustness was further reinforced through validation studies, demonstrating its ability to accurately interpolate and extrapolate under conditions and timelines not explicitly included in the training set. Given these results, our approach signifies broad applicability in complex industrial models across various fields, such as structural mechanics, convection–diffusion, and fluid dynamics, especially for problems with non-SPD Jacobians. This research represents a substantial stride in the field of hyper-reduction for nonlinear methods, indicating promising potential for further improvements in computational efficiency and model fidelity in the realm of reduced-order modeling. As an expansion, our methodology could further leverage advancements in local bases and neural networks, suggesting exciting avenues for future exploration and application.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request

Acknowledgments

The authors acknowledge financial support from the Spanish Ministry of Economy and Competitiveness, through the "Severo Ochoa Programme for Centres of Excellence in R&D" (CEX2018-000797-S)". We acknowledge financial support from the Generalitat de Catalunya through the FI_SDUR-2021 grant, which provided funding for the predoctoral training of Sebastian Ares de Parga Regalado. This project has received funding from the European High-Performance Computing Joint Undertaking (JU) under grant agreement Nos 955558 and 956104. The JU receives support from the European Union's Horizon 2020 research and innovation programme and from Spain, Germany, France, Italy, Poland, Switzerland, and Norway. Spanish Ministry of Science and Innovation (MICINN) supports the project through the project PCI2021-121945. This publication is part of the R&D project PCI2021-121944, financed by MCIN/AEI/10.13039/501100011033 and by the "European Union NextGenerationEU/PRTR".

Appendix. Singular value decomposition

This appendix provides details on the singular value decomposition (SVD) method used to find a basis matrix $\mathbf{\Phi}$ that satisfies the conditions presented in the main text. The procedure involves computing a truncated SVD [23] (TSVD) of the matrix $\mathbf{A}^u$, which is the matrix to be decomposed, with a given relative tolerance $0 < \epsilon_u \leq 1$. The TSVD function is defined symbolically as follows:

$$[\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}] = \text{SVD}(\mathbf{A}^u, \epsilon_u), \tag{68}$$

where $\mathbf{U} \in \mathbb{R}^{n\times r}$ is the matrix of left singular vectors, $\mathbf{\Sigma} \in \mathbb{R}^{r\times r}$ is the matrix of positive singular values (diagonal), and $\mathbf{V} \in \mathbb{R}^{m\times r}$ is the matrix of right singular vectors. These matrices satisfy the following relationships:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T + \mathbf{E}, \qquad \|\mathbf{E}\|_F \leq \epsilon\|\mathbf{A}\|_F, \qquad \mathbf{U}^T\mathbf{U} = \mathbf{V}^T\mathbf{V} = \mathbf{I}_{r\times r}, \qquad \Sigma_{(i+1,i+1)} \geq \Sigma_{(i,i)} > 0, \tag{69}$$

where $\|\cdot\|_F$ denotes the Frobenius norm, $\mathbf{E}$ is the truncation error, and $\Sigma(i,i)$ denotes the ith largest singular value of $\mathbf{A}^u$. The basis matrix $\mathbf{\Phi}$ is then chosen as the matrix of the first n columns of $\mathbf{U}$:

$$\mathbf{A}^u = \underbrace{\mathbf{U}_n}_{\mathbf{\Phi}} \mathbf{\Sigma}_n \mathbf{V}_n^T + \mathbf{E}, \tag{70}$$

where $\mathbf{U}_n$ and $\mathbf{V}_n$ are the first n columns of $\mathbf{U}$ and $\mathbf{V}$, respectively, and $\mathbf{\Sigma}_n$ is the $n \times n$ diagonal matrix containing the n largest singular values of $\mathbf{A}^u$.

References

[1] L. Sirovich, Turbulence and the dynamics of coherent structures. I. Coherent structures, Quart. Appl. Math. 45 (1987).
[2] S. Balachandar, Turbulence, coherent structures, dynamical systems and symmetry, AIAA J. 36 (1998).
[3] A.C. Antoulas, An overview of approximation methods for large-scale dynamical systems, Annu. Rev. Control 29 (2005).
[4] N.N. Cuong, K. Veroy, A.T. Patera, Certified real-time solution of parametrized partial differential equations, Handbook of Materials Modeling, 2005.
[5] C.W. Rowley, T. Colonius, R.M. Murray, Model reduction for compressible flows using POD and Galerkin projection, Physica D 189 (2004).
[6] P. Benner, S. Gugercin, K. Willcox, A survey of projection-based model reduction methods for parametric dynamical systems, SIAM Rev. 57 (2015).
[7] K. Carlberg, C. Bou-Mosleh, C. Farhat, Efficient non-linear model reduction via a least-squares Petrov-Galerkin projection and compressive tensor approximations, Internat. J. Numer. Methods Engrg. 86 (2011).
[8] R. Everson, L. Sirovich, Karhunen–Loève procedure for gappy data, J. Opt. Soc. Amer. A 12 (1995).
[9] S. Chaturantabut, D.C. Sorensen, Nonlinear model reduction via discrete empirical interpolation, SIAM J. Sci. Comput. 32 (2010).
[10] D. Galbally, K. Fidkowski, K. Willcox, O. Ghattas, Non-linear model reduction for uncertainty quantification in large-scale inverse problems, Internat. J. Numer. Methods Engrg. 81 (2010).
[11] D. Ryckelynck, A priori hyperreduction method: An adaptive approach, J. Comput. Phys. 202 (2005).
[12] K. Carlberg, C. Farhat, J. Cortial, D. Amsallem, The GNAT method for nonlinear model reduction: Effective implementation and application to computational fluid dynamics and turbulent flows, J. Comput. Phys. 242 (2013).
[13] C. Farhat, T. Chapman, P. Avery, Structure-preserving, stability, and accuracy properties of the energy-conserving sampling and weighting method for the hyper reduction of nonlinear finite element dynamic models, Internat. J. Numer. Methods Engrg. 102 (2015).
[14] J.A. Hernández, M.A. Caicedo, A. Ferrer, Dimensional hyper-reduction of nonlinear finite element models via empirical cubature, Comput. Methods Appl. Mech. Engrg. 313 (2017).
[15] J.A. Hernández, A multiscale method for periodic structures using domain decomposition and ECM-hyperreduction, Comput. Methods Appl. Mech. Engrg. 368 (2020).
[16] M. Barrault, Y. Maday, N.C. Nguyen, A.T. Patera, An 'empirical interpolation' method: application to efficient reduced-basis discretization of partial differential equations, C. R. Math. 339 (2004).
[17] C. Farhat, P. Avery, T. Chapman, J. Cortial, Dimensional reduction of nonlinear finite element dynamic models with finite rotations and energy-based mesh sampling and weighting for computational efficiency, Internat. J. Numer. Methods Engrg. 98 (2014).
[18] D. Amsallem, M.J. Zahr, C. Farhat, Nonlinear model order reduction based on local reduced-order bases, Internat. J. Numer. Methods Engrg. 92 (2012).
[19] K. Carlberg, M. Barone, H. Antil, Galerkin v. least-squares Petrov–Galerkin projection in nonlinear model reduction, J. Comput. Phys. 330 (2017).
[20] S. Grimberg, C. Farhat, R. Tezaur, C. Bou-Mosleh, Mesh sampling and weighting for the hyperreduction of nonlinear Petrov–Galerkin reduced-order models with local reduced-order bases, Internat. J. Numer. Methods Engrg. 122 (2021).
[21] P.J. Blonigan, K.T. Carlberg, F. Rizzi, M. Howard, J.A. Fike, Model reduction for hypersonic aerodynamics via conservative lspg projection and hyper-reduction, in: AIAA Scitech 2020 Forum, 2020.
[22] Y.S. Shimizu, E.J. Parish, Windowed space–time least-squares Petrov–Galerkin model order reduction for nonlinear dynamical systems, Comput. Methods Appl. Mech. Engrg. 386 (2021).

[23] C. Eckart, G. Young, The approximation of one matrix by another of lower rank, Psychometrika 1 (1936).
[24] M.J. Zahr, Adaptive Model Reduction to Accelerate Optimization Problems Governed by Partial Differential Equations (Ph.D. thesis), Stanford University, 2016.
[25] Y. Saad, Iterative methods for sparse linear systems, second ed., Methods (2003).
[26] S.S. An, T. Kim, D.L. James, Optimizing cubature for efficient integration of subspace deformations, ACM Trans. Graph. 27 (2008).
[27] F. Fang, C.C. Pain, I.M. Navon, A.H. Elsheikh, J. Du, D. Xiao, Non-linear Petrov-Galerkin methods for reduced order hyperbolic equations and discontinuous finite element methods, J. Comput. Phys. 234 (2013).
[28] A.H.J. Donea, Finite Element Methods for Flow Problems, 2003.